Final Year Design Project

E-Masjid System

By

Dawood Ahmed 2022-KS-158

Haris Ehsan 2022-KS-190

Under the supervision of

Sir Muhammad Kamran

Bachelor of Science in **Information Technology** ***(2022-2026)***

FACULTY OF COMPUTING &

INFORMATION TECHNOLOGY (FCIT),

UNIVERSITY OF THE PUNJAB, LAHORE.

E-Masjid System

A project presented to

University of the Punjab, Lahore

In partial fulfilment

of the requirement for the degree of

Bachelors of Science in **Information Technology** ***(2022-2026)***

By

Dawood Ahmed	**2022-KS-158 | 2022-089264**
Haris Ehsan	**2022-KS-190 | 2022-089301**

FACULTY OF COMPUTING &

INFORMATION TECHNOLOGY (FCIT),

UNIVERSITY OF THE PUNJAB, LAHORE

DECLARATION

We hereby declare that this software, neither whole nor as a part has been copied out from any source. It is further declared that we have developed this software and accompanied report entirely on the basis of our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other, we will stand by the consequences. No portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

Signature: ----------

Student Name 1[089264]

Signature: ----------

Student Name 2 [089301]

CERTIFICATE OF APPROVAL

It is to certify that the final year design project (FYDP) of BSIT "E-Masjid System" was developed by

Dawood Ahmed (2022-KS-158) and Haris Ehsan (2022-KS-158) under the supervision of "**Sir Muhammad Kamran**" in my opinion; it is fully adequate, in scope and quality for the degree of Bachelors of Science in **Information Technology.**

Signature: ----------

FYDP Supervisor:

Signatures (Faculty Advisory Committee (FAC)

Signatures			
Name			
	FAC1	FAC2	FAC3

Signature: ----------

Head of FYDP Coordination Office:

Signature:---------- Dated: _____

Chairperson, Department of Information Technology

Executive Summary

The E-Masjid System is a web based system and is built to help mosques to manage their daily activities in a digital way. In many places, mosques are still using manual registers for the records of donations and expenses and announcements are made only through loudspeakers which reach people within a limited area. This creates problems like lack of transparency, difficulties in keeping records, inconvenience for people who want to donate online. Also people have to visit the mosque physically to meet the religious scholar and tell the date of the nikah ceremony. We are building this project in order to fix these problems by developing an online system which will be easy to use.

The system allows the mosque admin to control the times of prayers, post announcements and also maintain proper records of donations and expenses. Community members are able to donate online through Stripe, view updates from the mosque, register for events and book Nikah services without physically visiting the mosque. We built a system to make sure Zakat and Sadaqah money is handled the right way. People who need help can now fill out a request, and a committee of trusted members looks at each one to decide who gets the funds. The system also works for several mosques at once. A Mosque Manager can turn different features on or off for each mosque. On the homepage, we added a section with photos and news about events to get the community attention. The system is designed to be simple and user friendly for all technical and not technical users.

Keywords: Mosque Management System, Donation Transparency, Multi Mosque Support

Acknowledgement

We would like to express our sincere thanks to **Sir Muhammad Kamran**, Lecturer at Government Graduate College, Civil Lines, Sheikhupura, for his valuable guidance, encouragement and support during the preparation of our project documentation. We are also thankful to our friends and families for their quiet support and motivation, which helped us stay focused throughout this phase of our work.

Signature: ----------

Dawood Ahmed [089264]

Signature: ----------

Haris Ehsan [089301]

Abbreviations

Table 1 Abbreviations

API	Application Programming Interface
FCIT	Faculty of Computing and Information Technology
FR	Functional Requirement
HTTP	HyperText Transfer Protocol
JWT	JSON Web Token
MERN	MongoDB, Express.js, React.js, Node.js
SMTP	Simple Mail Transfer Protocol
SSL	Secure Sockets Layer
UC	Use Case
UI	User Interface
URL	Uniform Resource Locator

Table of Contents

List of Tables

Tables of Figures

Chapter No 1
Project Proposal

1.1. Introduction

This chapter is an introduction about our Final Year Project E-Masjid System. We explain the reason why we chose this project and what problems will be solved. We also list the tools that we will be using and how our team members will be working together on this project. This chapter gives the complete overview of our project idea.

1.2. Background

In our country, mosques are the place where parent send their children to get Islamic teaching related to our religion. Person of any age can come to mosque and ask questions for which they have doubt in their mind so the person get the answers according to the Islam. Every town has one or more mosques that handles prayer schedule, organize religious events and collect donations. These days mosques manage their records in registers which become messy and can be destroyed by various causes. Also it is very time taking process to find the records on urgent bases. So there is a need of digital system where the records will be easily accessible and management of operations become easy.

1.3. Problem Statement

People who donate to the mosque do not know where their donations are used and how they are used. When families need nikah service, they have to visit physically to get nikah registrar. People who are away from their area mosque do not get the updates what announcement and events are going to happen. Right now, many mosques does not have a clear way to handle Zakat and Sadaqah requests. People who need help do not know where to go and the money is often given out without any real records. Another big problem is using paper registers for keeping records. This method is slow and if register get lost or damaged then all important information is gone forever.

1.4. Stakeholders & Interests

Stakeholder	Description	Interest
Mosque Administration	People who manage mosque operations like imam and committee members	Good management of operations and transparent record keeping
Donors	Community people who give money to mosque for different causes	Send their donations and see how funds are being used
Community Members	Local people who regularly visit mosque for prayers and activities	View prayer times, announcements, and events, register for events, request services easily.

Religious Scholars	Qualified Islamic scholars who perform nikah ceremonies and religious duties	Manage availability for Nikah, share special prayer timings, avoid double booking.
Committee Member	People who are committee member of the masjid and approve or reject the zakat request.	Approve or reject the zakat request.
Mosque Manager	Mosque manager manage different mosques and modules for each mosque	Create account of mosque and assign modules

1.5. Objectives:

1 To make a web based system for mosque management using MERN stack.

2 To manage donations in a clear way and record of where money is spent. Also people can donate online.

3 To show daily prayer times and special timings for Jummah and Ramadan.

4 To make an event section for Islamic classes, charity and other community programs.

5 To create an online system for booking nikah registrar for nikah.

6 To give different access to admin, religious scholar and normal users for security and management.

7 To implement multiple mosque system and to give access of features for each mosque.

8 To develop a Zakat and Sadaqah request system with committee review and committee members can approve or reject the request.

1.6. Scope:

In-scope

1. Digital management of prayer times and announcements.
2. Transparent tracking and record keeping of donations.
3. Create an event and registration system.
4. Online booking system for Nikah services.
5. User authentication for admin, religious scholar and community members.
6. Single mosque management system.
7. Responsive web design for mobile and desktop.
8. Online payment gateway integration.
9. Multi mosque management system.
10. Needy user can send fund request and committee member can see and approve or reject the request.

Out-of-scope

1. Mobile app
2. Automated SMS or email notification system

1.7. Assumptions

Following are the assumptions of our system:

1. Mosque management know basic computer operations like using a browser and typing.
2. Users have internet connection and email addresses.
3. The mosque has at least one computer or laptop for the admin to use.
4. Religious scholars can use simple websites and click buttons.
5. Community members can use web browsers on their phones or computers.
6. People will use the system honestly and enter correct information.

1.8. Risks

Following are the risks that can occur in the development of our project:

1. **Payment security issues**: Someone may attempt to steal payment information.
   Mitigation: We will use Stripe payment gateway which is very secure and helps to follow international security standards.
2. **System downtime during prayer times:** If system stops working when people need to check prayer times
   Mitigation: We will use a reliable hosting service and will maintain backup of data.
3. **Elderly users finding system difficult:** Old people might not understand how to use website.
   Mitigation: We are going to design simple interface with big buttons and text to be clear.
4. **Data loss due to system crash:** If database gets damaged, all the records can be lost.
   Mitigation: We will setup automatic weekly backup of database to cloud storage.

1.9. Success Criteria

For the success of our system , the following features must work properly:

Table 2 Project Success Criteria

User Registration and Login of admin , religious scholar and community members.
Donation recording and tracking system.

Prayer times management and display.
Event creation and registration.
Nikah booking system.
Financial reports showing income and expenses.
Responsive design to work on mobile phones.
Safe payment processing using Stripe.
Admin Dashboard to manage all the operations.
Simple and easy to use interface.
Data back up .
System deployment on live server.
Multi mosque system working
User can request fund for zakat

1.10. Tools, Libraries & Technologies

Table 3 Tools Technologies and Libraries

Tools, Libraries, And Technologies	**Tools**	**Version**	**Rationale**
	VS Code , MS Word, Draw.io	Latest	code editor with good features
	Libraries	**Version**	**Rationale**
	React.js , Express.js JWT , Mongoose Nodemailer	Latest	For building responsive design and creating APIs and handling server side logic.
	Technology	**Version**	**Rationale**
	React , Nodejs , JS , Mongo DB, Stripe API	Latest	To build a complete web app

1.11. Work Division

Table 4 Project Team Members Work Division

Sr.	Roll Number	Name	Role Assignment & Work Division

No			
1.	089264	Dawood Ahmed	Backend
2.	089301	Haris Ehsan	Frontend

1.12. Conclusion

In this chapter, we introduced our project idea of E-Masjid System. We explained the issues with the current management of mosques and how our digital solution will help. We specified clear objectives and scope of our project. We also identified stakeholders and discussed tools we will be using.

Chapter No 2
Literature Review

2.1 Literature Survey

We looked at how mosques currently manage their work. We saw that most mosques in Pakistan use paper registers for keeping records. We also searched online and found some mosque management systems but they are mostly made for other countries. These systems do not match the needs of Pakistani mosques like handling local nikah booking or showing expenses in a simple way that our community trusts.

2.2 Related Work:

There are some systems which are being used for churches that handle donation, events and prayer schedule. In Pakistan most mosque still use old methods like keeping records in register or making announcements on speaker. Most existing Islamic applications focus only on prayer times, Quran reading and Qibla direction, they don't help in mosque management.

2.3 Gap Analysis:

After checking existing solutions, we found several important gaps Firstly, we have seen that no system has been made for Pakistani mosque that handle situations like nikah registrar booking. Secondly, the current system does not give the financial transparency that people want. Our project fills these gaps with a cheap and easy to use platform.

2.4 Summary

In this chapter, we have observed how mosques work today. We found that most use old methods that have problems. Our system will solve these problems by giving one platform for all mosque work. This chapter gave us an idea about the features that we need to add in our project.

Chapter 3

Requirements Analysis

1. Analysis

This section explains the detailed requirements for the E-Masjid System. The main purpose of this SRS is to explain what our system will do and who will use it and what functions it will perform. This analysis helps us to understand that what features to build and how they should work for different types of users.

3.1. User classes and characteristics

Table 5 User classes

User Class	User Characteristics
Mosque Manager	This is the super admin of the whole system. He can give access to any masjid and create admin accounts for those masjids and decide which features each masjid can use.
Mosque Administration	These are the admins of a single masjid. People who manage mosque operations like imam and committee members. They need full control over system and ability to manage all activities.
Committee Member	A trusted person selected by the masjid admin to review zakat and sadaqah requests. Committee members receive email notifications when a new request comes, log in to a dashboard and approve or reject requests with a reason.
Donor	People who give donations to the mosque. They can view donation records and reports. They may donate online or in person.
Community Members	Local people who visit mosque regularly. They need to see prayer times, announcements, events and request services. They have limited access.
Religious Scholars	Islamic scholars who perform nikah ceremonies. They need to manage their availability and see their booking schedule.

3.2. Requirement Identifying Technique

To find out what our E-Masjid System should do, we used different methods to understand what users really need. First, we talked to imam and listened to the problems related to mosque. They told us about manual record keeping, lack of transparency in donations, zakat or sadaqah handling and how difficult it is to manage events. Then we used the Use Case technique because our system is an interactive website where different users do different things. This helped us understand how mosque manager, mosque admins, committee members, religious scholars and community members will use the system. We made use case diagrams to show all the main features clearly.

3.3. Functional Requirements

We identified 14 main functional requirements for our system. Each feature has specific functional requirements that explain what the system should do. These requirements are written from users view to clearly explain the expected behavior.

Functional Requirement 1

Table 6 Functional Requirement 1

Identifier	FR-1
Title	User Registration and Login
Requirement	The system will allow users to register and login with email and password, with different access levels for admin, Religious Scholar and community members.
Source	System security needs
Inputs	Email, password, name, phone number, user role
Destination	User data is stored in database and login result is shown on screen
Outputs	Successful registration and login message or error message
Rationale	To protect sensitive information and manage permissions
Business Rule	Admin users have full access, community users and religious scholar have limited access
Dependencies	None
Priority	High

Functional Requirement 2

Table 7 Functional Requirement 2

Identifier	FR-2
Title	Record Donations
Requirement	The mosque admin will be able to record cash donations with donor name, amount, date, and donation type. A donation can be marked as anonymous by the donor.
Source	Mosque committee discussion

Inputs	Donor name, amount, date, donation type
Destination	Saved in donations collection in database
Outputs	Donation record created and success message shown
Rationale	To maintain proper records and show transparency
Business Rule	Each donation must have at least donor name and amount
Dependencies	FR-1
Priority	High

Functional Requirement 3

Table 8 Functional Requirement 3

Identifier	FR-3
Title	Record Mosque Expenses
Requirement	Admin can add where mosque money is spent like for repairs, electricity etc.
Source	Discussions with mosque imam about community trust issues.
Inputs	Expense description, amount, date, category
Destination	Saved in expenses collection
Outputs	Expense record added successfully
Rationale	Community wants to see how their donations are being used
Business Rule	Every expense must have description, amount, and date
Dependencies	FR-1
Priority	High

Functional Requirement 4

Table 9 Functional Requirement 4

Identifier	FR-4

Title	Show Donation and Expense Reports
Requirement	The system will show donation records, expense details, and financial reports so people can see both income and spending.
Source	Discussions with mosque imam about community trust issues.
Inputs	Selection of report date or month
Destination	Displayed on admin and user dashboard
Outputs	List of donations, list of expenses, total income, total expenses
Rationale	People want to see where their money is spent.
Business Rule	Reports should show income vs expenses clearly.
Dependencies	FR-2, FR-3
Priority	High

Functional Requirement 5

Table 10 Functional Requirement 5

Identifier	FR-5
Title	Event & Announcement Management
Requirement	The admin will be able to add, update, or remove events and announcements such as Islamic classes, community programs, and Eid prayers. Users can view them on the main page. The homepage includes a promotional section.
Source	Community engagement needs
Inputs	Event title, date, time, description, announcement details
Destination	Stored in database and shown on website homepage
Outputs	Event or announcement published successfully
Rationale	Helps mosque communicate better with community

Business Rule	Events should show date, time and location clearly
Dependencies	FR-1
Priority	Medium

Functional Requirement 6

Table 11 Functional Requirement 6

Identifier	FR-6
Title	Book Nikah Services
Requirement	Community members will be able to book nikah registrar for nikah ceremonies by selecting date and providing contact details
Source	Community needs this service
Inputs	Date, time, contact information, ceremony details
Destination	Saved in nikah bookings collection
Outputs	Booking request submitted and confirmation message
Rationale	People need easy way to arrange nikah registrar
Business Rule	Booking requests must include confirm date and contact information
Dependencies	FR-1
Priority	Medium

Functional Requirement 7

Table 12 Functional Requirement 7

Identifier	FR-7
Title	Manage Prayer Times
Requirement	The admin will be able to set and update daily prayer times including special timings for Jummah and Ramadan.
Source	Basic religion need

Inputs	Fajr, Zuhr, Asr, Maghrib, Isha times, special timings
Destination	Saved in prayer times collection
Outputs	Updated prayer times visible to everyone
Rationale	People need accurate prayer schedules.
Business Rule	Prayer times must be visible without login.
Dependencies	FR-1
Priority	High

Functional Requirement 8

Table 13 Functional Requirement 8

Identifier	FR-8
Title	Online Donation System
Requirement	Community members will be able to make donations online through the website. They will enter the amount and personal details, process payments securely using Stripe, and see a success message on the screen after the payment is processed. The donor can select an anonymous checkbox so his name is not displayed publicly.
Source	Community needs an easier way to donate
Inputs	Donation amount, card details, donor information
Destination	Saved in donations collection and sent to Stripe for processing
Outputs	Payment success or failure message and donation recorded
Rationale	People want to donate easily without visiting mosque
Business Rule	Each online donation must record donor information and amount
Dependencies	FR-1
Priority	Medium

Functional Requirement 9

Table 14 Functional Requirement 9

Identifier	FR-9
Title	Password Reset through Email
Requirement	If users forget their password, they can reset it using their email address.
Source	Basic every users and system need
Inputs	User email address
Destination	Email sent to user with reset link
Outputs	Password reset email sent and password change successful
Rationale	People often forget passwords and need an easy way to get back into their account
Business Rule	The reset link should work only for 24 hours and can be used one time
Dependencies	FR-1
Priority	High

Functional Requirement 10

Table 15 Functional Requirement 10

Identifier	FR-10
Title	Check Nikah Booking Status
Requirement	Users can see if their Nikah request is pending, accepted, or rejected
Source	Community members want to check their nikah request status
Inputs	User clicks on "My Nikah Bookings"
Destination	Displayed on user my booking page
Outputs	List of bookings with current status
Rationale	People want to know what is happening with their booking request
Business Rule	The status should update automatically when scholar changes it

Dependencies	FR-6
Priority	Medium

Functional Requirement 11

Table 16 Functional Requirement 11

Identifier	FR-11
Title	Create Scholar Accounts
Requirement	Admin can create special accounts for religious scholars who perform Nikah service
Source	Use Case Analysis
Inputs	Scholar name, email, phone, specialization
Destination	Saved in users collection with "scholar" role
Outputs	Scholar account created and scholar can login
Rationale	Scholars need their own accounts to manage Nikah requests
Business Rule	Only admin can create and manage these special accounts
Dependencies	FR-1
Priority	Medium

Functional Requirement 12

Table 17 Functional Requirement 11

Identifier	FR-12
Title	Multi Mosque Management
Requirement	A Mosque Manager can give software access to any masjid in Pakistan. He can create a masjid profile and decide which features are active for that masjid.
Source	Had a discussion with sir
Inputs	Masjid name, location, contact person, list of selected features

Destination	Saved in Masjids collection and features stored as a list of enabled features
Outputs	Masjid is created, admin account is linked and only enabled features appear for that masjid user
Rationale	We want to control multiple masjid access and the features they can access.
Business Rule	Only a Mosque Manager can create a masjid account. A masjid admin cannot change which features are enabled.
Dependencies	FR-1
Priority	High

Functional Requirement 13

Table 18 Functional Requirement 13

Identifier	FR-13
Title	Zakat and Sadaqah Request Submission
Requirement	A logged in community member can submit a request for financial help from the masjid zakat or sadaqah funds. He must provide his name, contact, reason for the request and the amount needed.
Source	Had a discussion with sir
Inputs	User name, phone, reason, requested amount, supporting details
Destination	Saved in zakat requests collection and email notification sent to all committee members of that masjid
Outputs	Request submitted message is shown to the user and request appear in committee dashboard
Rationale	There is no simple way to apply for zakat help. Many needy people do not know how to ask. This gives them a clear and private way..
Business Rule	Only logged in community members can submit a request. One user can submit multiple requests but each request is separate.
Dependencies	FR-1,FR-14
Priority	High

Functional Requirement 14

Table 19 Functional Requirement 14

Identifier	FR-14
Title	Committee Review of Zakat Requests
Requirement	The masjid admin creates committee member accounts. When a new zakat or sadaqah request arrives then all committee members get an email. They log into a dashboard, see the request details, discuss among themselves and then approve or reject the request with a written reason. The user can see the final decision..
Source	Had a discussion with sir
Inputs	Committee member login and decision with a comment
Destination	Decision saved in zakat requests collection and status updated and visible to user
Outputs	Request status changes to approved or rejected and reason is shown to user
Rationale	The community knows that requests were reviewed by real people
Business Rule	Only committee members can change the request status. Once approved or rejected then the decision cannot be changed.
Dependencies	FR-1, FR-13
Priority	High

3.4. Non-Functional Requirements

This section describes the quality requirements of our system that how it should perform and how easily we can use it and how secure it should be.

3.4.1 Reliability

The system must be reliable in the day to day running of the mosque. It should not crash frequently and should recover quickly if any problems occur.

1. The system must stay running during busy times like Jumuah and Ramadan.
2. If there is a problem, the system should recover quickly to avoid long pauses.
3. Donation data should not be lost even if system has problems
4. Important data should be automatically backed up after every week

5. Email notifications for zakat requests should be sent reliably. If the email service is temporarily unavailable then the request is still saved in the database and the committee can see it when they log in.

3.4.2 Usability

The system must be easy to use and less complex to the mosque admin and ordinary users. All buttons and forms will be clear and labeled properly.

1. Prayer times should be available to new users in 2 clicks
2. Donation recording process should take less than 3 minutes for admin
3. Nikah booking form should be completable within 5 minutes
4. Large fonts and clear buttons should be used by the elderly users in interface
5. Every key feature must be available at home page

3.4.3 Performance

When there are several people using the system, it should be fast and easily functioning.

1. Prayer times page should load within 3 seconds
2. Donation reports should generate within 5 seconds
3. System should handle up to 100 users at the same during Friday prayers
4. The registration of the event must take a maximum of 5 seconds
5. Email notifications to committee members should be sent within 30 seconds of a new request being submitted.

3.4.4 Security

The system should protect sensitive information like donor details and maintain privacy.

1. Database should store user passwords in an encrypted format
2. The system must not allow unauthorized access to administrator functions
3. Session should timeout after 1 hour of inactivity
4. All payment transactions must be secured using SSL encryption.

3.5. External Interface Requirements

This section describes how our E-Masjid system will interact with users and other systems. It covers the user interface design, software connections, and communication methods.

3.5.1 User Interfaces Requirements

We will have a clean and simple interface on our system which will be effective with the mosque users who are aged, administrators and community members.

Design Guidelines:

1. Use simple colors that are common in Islamic design
2. Large buttons and text for easy reading, especially for aged users
3. Consistent navbar menu on all pages
4. Prayer times always visible on the header
5. Mobile friendly design that works on smartphones and tablets
6. Use common icons that people can easily understand
7. Error messages in simple language, not technical terms

Layout Standards:

1. Homepage shows prayer times, announcements, and quick access to main features
2. Admin dashboard with clear sections for donations, events, and services
3. Forms should be simple with clear labels and instructions
4. Use responsive design that adjusts to different screen sizes

3.5.2 Software interfaces

The system will use the following software tools.

Frontend:

1. React.js web application running in modern browsers
2. Works on iPhones and Android phones

Backend:

1. Node.js server with Express.js framework
2. MongoDB database for storing all data
3. JWT tokens for user authentication

External Services:

1. Stripe payment gateway will be used for real online donations
2. Email service for sending password reset links and fund requests emails to the committee members.

3.5.3 Hardware interfaces

The system will run on any normal computer or smartphone that has an internet connection and a browser. No special hardware is required, but a basic server will host the system.

3.5.4 Communications interfaces

Our system will use web communication:

Network Requirements:

1. Web access through Standard HTTP/HTTPS
2. Internet connection needed to operate the system
3. No special network configuration needed

Communication Features:

1. Basic website notifications for new announcements
2. No SMS integration initially
3. No email marketing system
4. Simple contact forms for communication

3.6 Use case Analysis

Use Case #1 Register User

Table 20 Use Case 1

UC Identifier	UC1
Use Case Name	Register User
Requirements Traceability	FR-1
Purpose	To allow new users to register in the system using email and password.
Priority	High
Preconditions	User is not already registered.
Post conditions	New user account created.
Actors	Community Member
Extends	None
Main Success Scenario	1. User opens registration page. 2. Enters name, email, phone no, password. 3. System validates input. 4. Account created successfully. 5. User can now login to the system
Alternate Flows	User already exists then system shows “Email already registered.”
Exceptions	Invalid input fields or server error.

Includes	None

Use Case #2 Login User

Table 21 Use Case 2

UC Identifier	UC2
Use Case Name	Login User
Requirements Traceability	FR-1
Purpose	To let registered users log in using email and password.
Priority	High
Preconditions	User must be registered.
Post conditions	User successfully logged in
Actors	Community Member
Extends	None
Main Success Scenario	1. User enters email and password. 2. System checks credentials. 3. If correct, user is logged in and taken to homepage.
Alternate Flows	If wrong password then system show “Invalid email or password.”
Exceptions	Server not responding
Includes	Forgot Password

Use Case #3 Forgot Password

Table 22 Use Case 3

UC Identifier	UC3
Use Case Name	Forgot Password
Requirements Traceability	FR-9
Purpose	To help users who forgot their password
Priority	High
Preconditions	User must be registered with valid email
Post conditions	User can set new password and login
Actors	Community Member
Extends	Login User
Main Success Scenario	1. User clicks "Forgot Password" on login page 2. Enters email address 3. System sends password reset link to email 4. User clicks link in email 5. Enters new password

	6. System updates password 7. User can now login with new password
Alternate Flows	If email not found, system still shows "If email exists, reset link sent" for security
Exceptions	If email service is not working
Includes	User authentication

Use Case #4 View Prayer Times

Table 23 Use Case 4

UC Identifier	UC4
Use Case Name	View Prayer Times
Requirements Traceability	FR-7
Purpose	To let users see daily prayer timings
Priority	High
Preconditions	None
Post conditions	User can see all prayer times
Actors	Community Member
Extends	None
Main Success Scenario	1. User visits website homepage 2. Prayer times are displayed on top of page 3. User can see Fajr, Zuhr, Asr, Maghrib, Isha times 4. Special timings for Jummah are also shown on prayer times page
Alternate Flows	User can view weekly prayer schedule
Exceptions	If admin hasn't updated times, show default times
Includes	None

Use Case #5 View Events & Register

Table 24 Use Case 5

UC Identifier	UC5
Use Case Name	View Events & Register
Requirements Traceability	FR-5
Purpose	To see mosque events and register for them
Priority	Medium
Preconditions	User must be logged in to register
Post conditions	User registered for event
Actors	

	Community Member
Extends	None
Main Success Scenario	1. User goes to Events page 2. Views list of upcoming events 3. Clicks on event to see details 4. Clicks "Register for Event" 5. System confirms registration 6. User gets confirmation message
Alternate Flows	User can view events without login, but needs login to register
Exceptions	If system cannot register user due to server issue
Includes	Login User

Use Case #6 View Donation Records

Table 25 Use Case 6

UC Identifier	UC6
Use Case Name	View Donation and Expense Records
Requirements Traceability	FR-4
Purpose	To see donation reports and where money is spent
Priority	High
Preconditions	User must be logged in to view reports
Post conditions	User can see financial reports
Actors	Community Member
Extends	None
Main Success Scenario	1. User goes to donation report page 2. Views donation records and amounts 3. Sees expense details like "5000 for new ceiling fans" 4. Builds trust in mosque management
Alternate Flows	User can filter reports by date filter
Exceptions	If no data available, show "No records yet"
Includes	Login User

Use Case #7 View Announcements

Table 26 Use Case 7

UC Identifier	UC7
Use Case Name	View Announcements

Requirements Traceability	FR-5
Purpose	To read important mosque announcements
Priority	Medium
Preconditions	None
Post conditions	User reads announcements
Actors	Community Member
Extends	None
Main Success Scenario	1. User visits website homepage 2. Announcements are shown in main section 3. User reads important updates 4. Urgent announcements are highlighted in red
Alternate Flows	User can see old announcements
Exceptions	If no announcements, show "No current announcements"
Includes	None

Use Case #8 Make Donation

Table 27 Use Case 8

UC Identifier	UC8
Use Case Name	Make Online Donation
Requirements Traceability	FR-8
Purpose	To donate money to mosque online
Priority	Medium
Preconditions	User must be logged in
Post conditions	Donation processed and recorded
Actors	Community Member, Donor
Extends	None
Main Success Scenario	1. User clicks "Donate Online" 2. Selects donation type 3. Enters amount in rupees 4. Enters card details for Stripe payment 5. Clicks "Donate Now" 6. Stripe processes payment 7. System shows "Donation Successful"
Alternate Flows	If user is not logged in, system redirects user to Login page before allowing donation.
Exceptions	If payment fails then show "Payment failed try again"
Includes	Login User

Use Case #9 Book Nikah Services

Table 28 Use Case 9

UC Identifier	UC9
Use Case Name	Book Nikah Services
Requirements Traceability	FR-6
Purpose	To book religious scholar for marriage ceremony
Priority	Medium
Preconditions	User must be logged in
Post conditions	Nikah booking request submitted
Actors	Community Member
Extends	None
Main Success Scenario	1. User goes to Nikah Services page 2. Selects preferred date and time 3. Fills contact details and ceremony information 4. Submits the request 5. System sends confirmation "Request Submitted"
Alternate Flows	User can view their previous bookings
Exceptions	If date not available, show "Please select another date"
Includes	Login User

Use Case #10 View Booking Status

Table 29 Use Case 10

UC Identifier	UC10
Use Case Name	View Booking Status
Requirements Traceability	FR-10
Purpose	To check if Nikah booking request is accepted or pending
Priority	Medium
Preconditions	User must be logged in and have booking request
Post conditions	User sees current status of their request
Actors	Community Member
Extends	Book Nikah Services
Main Success Scenario	1. User goes to My Bookings page 2. Views list of their Nikah requests 3. See status like Pending, Accepted, or Rejected 4. If accepted, sees confirmed date and time 5. If rejected, sees reason if provided

Alternate Flows	User can cancel pending requests
Exceptions	If no bookings, show "No booking requests yet"
Includes	Login User

Use Case #11 Admin Login

Table 30 Use Case 11

UC Identifier	UC11
Use Case Name	Admin Login
Requirements Traceability	FR-1
Purpose	To let mosque admin access the admin dashboard
Priority	High
Preconditions	Admin must have admin account details
Post conditions	Admin gains access to admin dashboard
Actors	Mosque Admin
Extends	None
Main Success Scenario	1. Admin goes to special /admin/login URL 2. Enters admin username and password 3. Clicks "Admin Login" 4. System verifies admin details 5. Admin dashboard opens with all management options
Alternate Flows	If wrong details, show "Invalid admin login"
Exceptions	If admin account not set up yet
Includes	User Authentication

Use Case #12 Manage Donations & Expenses

Table 31 Use Case 12

UC Identifier	UC12
Use Case Name	Manage Donations & Expenses
Requirements Traceability	FR-2, FR-3
Purpose	To handle all money records like donations and expenses
Priority	High
Preconditions	Admin must be logged in
Post conditions	Financial records updated
Actors	Mosque Admin
Extends	None
Main Success Scenario	1. Admin goes to Manage Donations & Expenses section

	2. Can add new cash donations with donor details 3. Can record expenses like "5000 for mosque repairs" 4. Can edit or delete existing records 5. System updates financial reports automatically
Alternate Flows	Admin can generate monthly financial reports
Exceptions	If invalid data entered, show appropriate error messages
Includes	Admin Login

Use Case #13 Manage Prayer Times

Table 32 Use Case 13

UC Identifier	UC13
Use Case Name	Manage Prayer Times
Requirements Traceability	FR-7
Purpose	To set and update daily prayer timings for community
Priority	High
Preconditions	Admin must be logged in
Post conditions	New prayer times displayed on website
Actors	Mosque Admin
Extends	None
Main Success Scenario	1. Admin goes to Prayer Times section 2. Updates all five daily prayer times 3. Sets special Jummah prayer timings 4. Clicks "Save Times" 5. New times immediately show on website for everyone
Alternate Flows	Admin can set weekly schedule instead of daily updates
Exceptions	If invalid time format, show "Please use correct time format"
Includes	Admin Login

Use Case #14 Manage Events

Table 33 Use Case 14

UC Identifier	UC14
Use Case Name	Manage Events
Requirements Traceability	FR-5
Purpose	To create, update, and manage mosque events
Priority	Medium
Preconditions	Admin must be logged in
Post conditions	Events published and visible to community

Actors	Mosque Admin
Extends	None
Main Success Scenario	1. Admin goes to Events Management section 2. Creates new events with details and registration options 3. Updates existing event information 4. Deletes or cancels events when needed 5. Events appear on website for community registration
Alternate Flows	Admin can save events as drafts before publishing
Exceptions	If past date entered, show "Event date cannot be in past"
Includes	Admin Login

Use Case #15 Manage Announcements

Table 34 Use Case 15

UC Identifier	UC15
Use Case Name	Manage Announcements
Requirements Traceability	FR-5
Purpose	To post and manage important mosque announcements
Priority	Medium
Preconditions	Admin must be logged in
Post conditions	Announcements visible on website
Actors	Mosque Admin
Extends	None
Main Success Scenario	1. Admin goes to Announcements section 2. Creates new announcements with titles and details 3. Updates existing announcements if information changes 4. Deletes old or incorrect announcements 5. Marks announcements as urgent if important 6. Announcements show on website immediately
Alternate Flows	Admin can schedule announcements for future dates
Exceptions	If announcement too long, show "Please shorten announcement"
Includes	Admin Login

Use Case #16 Manage Religious Scholar Account

Table 35 Use Case 16

UC Identifier	UC16
Use Case Name	Manage Religious Scholar Account
Requirements	FR-11

Traceability	
Purpose	To create and manage account for Nikah service scholars
Priority	Medium
Preconditions	Admin must be logged in
Post conditions	Scholar accounts created and activated for Nikah management
Actors	Mosque Admin
Extends	None
Main Success Scenario	1. Admin goes to Scholar Management section 2. Creates new scholar accounts with login details 3. Updates scholar account details if needed 4. Deletes or deactivates scholar accounts 5. Scholars receive login details and can manage Nikah requests
Alternate Flows	Admin can reset scholar passwords if forgotten
Exceptions	If email already used, show "Email already registered"
Includes	Admin Login

Use Case #17 Check Nikah Requests

Table 36 Use Case 17

UC Identifier	UC17
Use Case Name	Check Nikah Requests
Requirements Traceability	FR-6
Purpose	To allow religious scholars to check new nikah booking requests and update their status as accepted or rejected.
Priority	Medium
Preconditions	The religious scholar account must already be created by the mosque admin, and the scholar must be logged into the system.
Post conditions	The nikah request status is updated to either “Accepted” or “Declined,” and the request status is updated in the admin dashboard.
Actors	Religious Scholar, Mosque Admin
Extends	None
Main Success Scenario	1.Religious scholar logs into system 2.Goes to "Nikah Requests" page 3.Views list of pending booking requests 4.Clicks on a request to see details 5.Clicks "Accept" or "Decline" button 6.System updates request status 7.Admin sees updated status in dashboard

Alternate Flows	If no requests available, system shows "No pending requests" If scholar tries to accept conflicting time, system shows "Time not available"
Exceptions	If system error occurs, shows "Unable to update request"
Includes	User authentication

Use Case #18 Submit Zakat or Sadaqah Request

Table 37 Use Case 17

UC Identifier	UC18
Use Case Name	Submit Zakat or Sadaqah Request
Requirements Traceability	FR-13
Purpose	To allow a community member to submit a request for financial help from Zakat or Sadaqah funds
Priority	High
Preconditions	User must be logged in
Post conditions	Request saved and committee members notified by email
Actors	Community Member
Extends	None
Main Success Scenario	1. User clicks "Request Zakat Help" on home page. 2. Fills name, phone, reason, amount. 3. Clicks Submit. 4. System saves request with status "Pending". 5. System sends email to all committee members. 6. User sees "Request submitted successfully."
Alternate Flows	If any field empty, system shows "Please fill all fields."
Exceptions	If email service fails, request is still saved and committee can see it on dashboard.
Includes	User authentication

Use Case #19 Review Zakat Request

Table 38 Use Case 17

UC Identifier	UC19
Use Case Name	Review Zakat Request
Requirements Traceability	FR-14
Purpose	To allow committee members to review pending Zakat requests and approve or reject them

Priority	High
Preconditions	Committee member must be logged in. At least one pending request exists.
Post conditions	Request status updated to Approved or Rejected. Requester can see the decision.
Actors	Committee Member
Extends	None
Main Success Scenario	1. Committee member logs in. 2. Opens review dashboard. 3. Clicks on a pending request to see details. 4. Clicks "Approve" or "Reject". 5. Writes a reason. 6. Submits decision. 7. Request status updates. 8. Requester sees new status on their "My Requests" page.
Alternate Flows	If request already reviewed, shows "Request already reviewed."
Exceptions	If system error, shows "Unable to update request."
Includes	User authentication

Use Case #20 Manage Mosques

Table 39 Use Case 17

UC Identifier	UC20
Use Case Name	Manage Mosques
Requirements Traceability	FR-12
Purpose	To allow Mosque Manager to create new mosque profiles and assign modules
Priority	High
Preconditions	Mosque Manager must be logged in
Post conditions	New mosque created with admin account. Modules assigned as selected.
Actors	Mosque Manager
Extends	None
Main Success Scenario	1. Mosque Manager logs in. 2. Goes to "Manage Mosques". 3. Clicks "Add New Mosque". 4. Fills name, location, contact details. 5. Selects which modules to enable. 6. Clicks Save. 7. System creates mosque record and admin account. 8. New mosque appears in the list.

Alternate Flows	Manager can edit existing mosque module settings.
Exceptions	If mosque name already exists, shows "Mosque already registered."
Includes	User authentication

Use Case #21 Manage Committee Members

Table 40 Use Case 17

UC Identifier	UC21
Use Case Name	Manage Committee Members
Requirements Traceability	FR-14
Purpose	To allow mosque admin to create and manage committee member accounts for Zakat review
Priority	Medium
Preconditions	Admin must be logged in
Post conditions	Committee member account created and linked to mosque
Actors	Mosque Admin
Extends	None
Main Success Scenario	1. Admin goes to Committee Management section. 2. Clicks "Add Committee Member". 3. Fills name, email, phone. 4. System creates account with role "committee" and links to mosque. 5. New member appears in list.
Alternate Flows	Admin can remove or deactivate committee members.
Exceptions	If email already used, shows "Email already registered."
Includes	User authentication

Use Case Diagram

These diagrams shows all the users and features of our E-Masjid system. It helps show how different users interact with different parts of the system

.

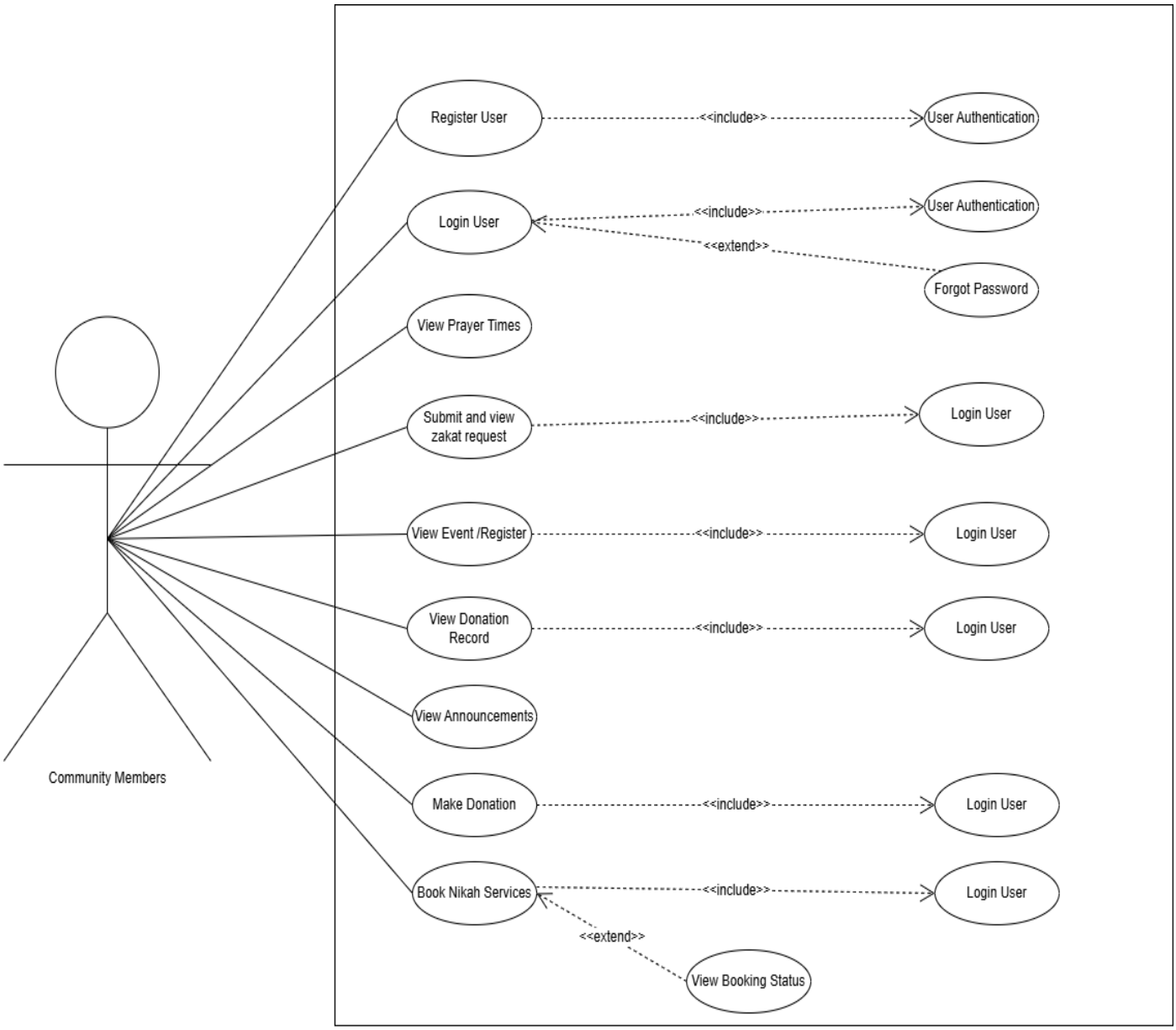

Figure 1 Use Case diagram of Community members

Figure 2 Use Case diagram of Mosque Admin

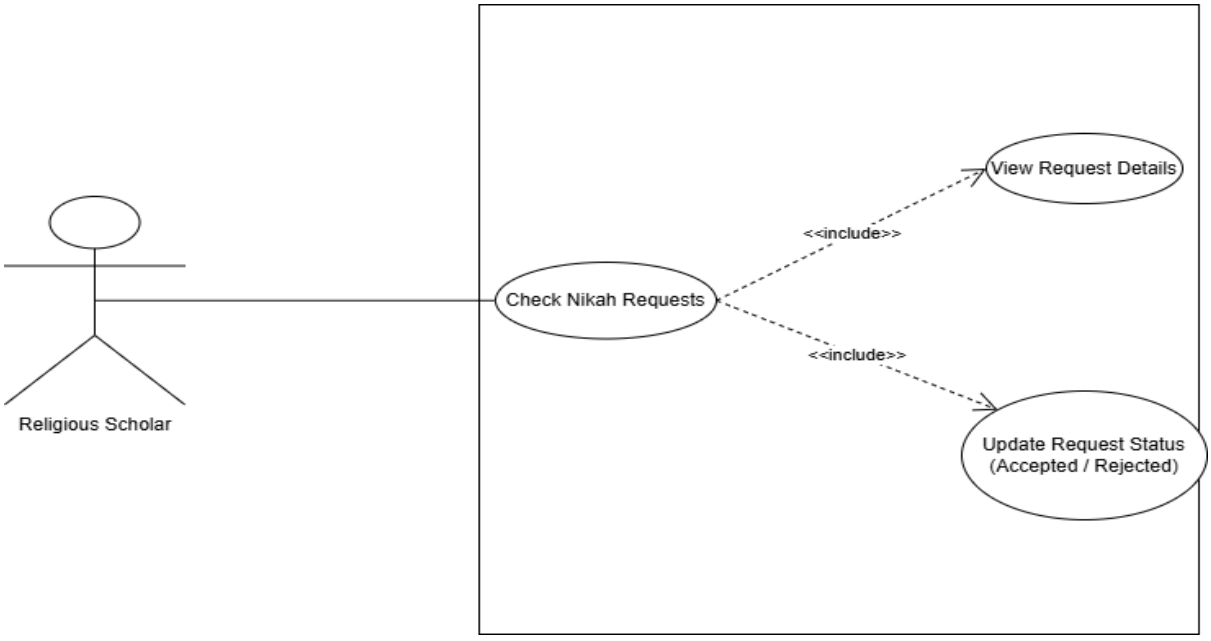

Figure 3 Use Case diagram of Religious Scholar

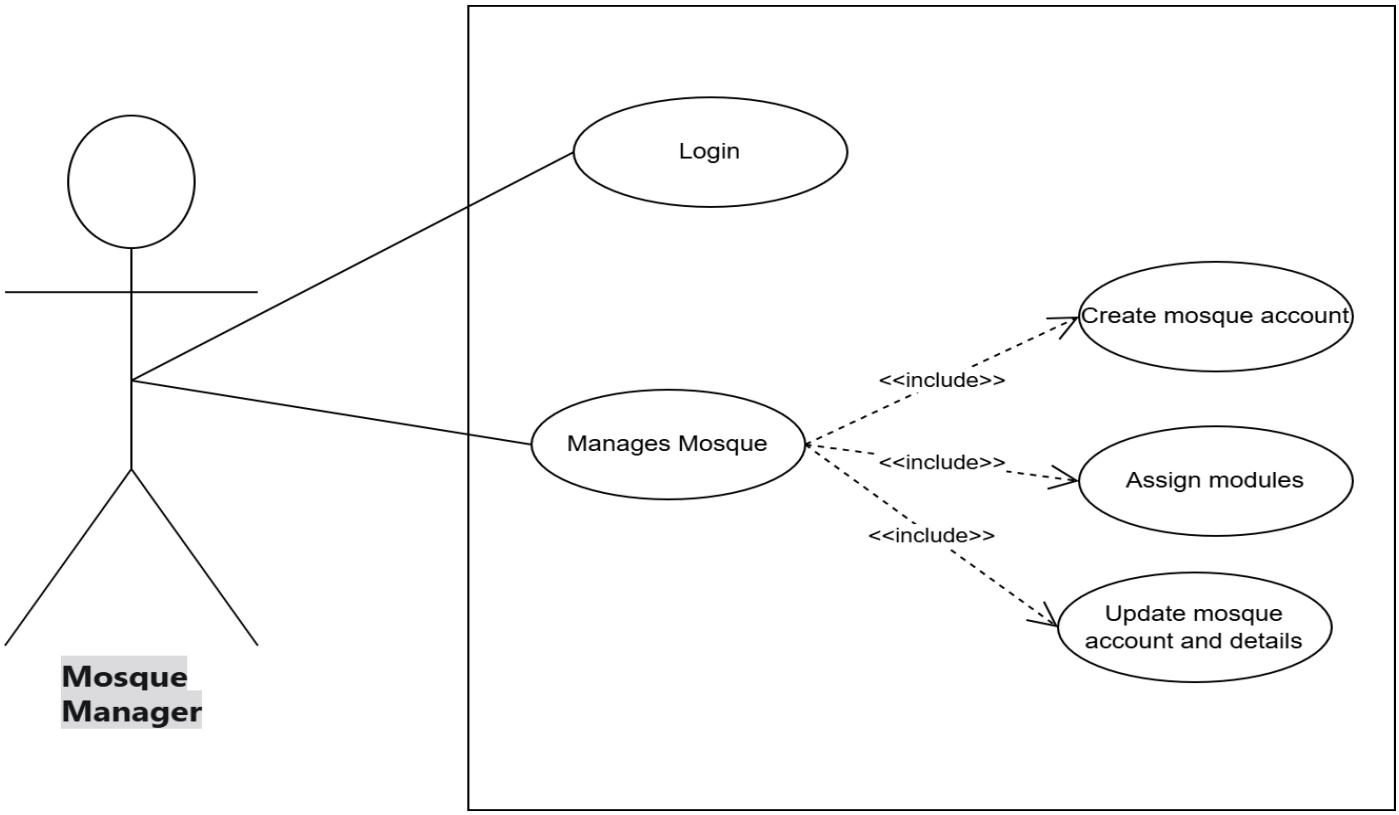

Figure 4 Use case diagram of Mosque Manager

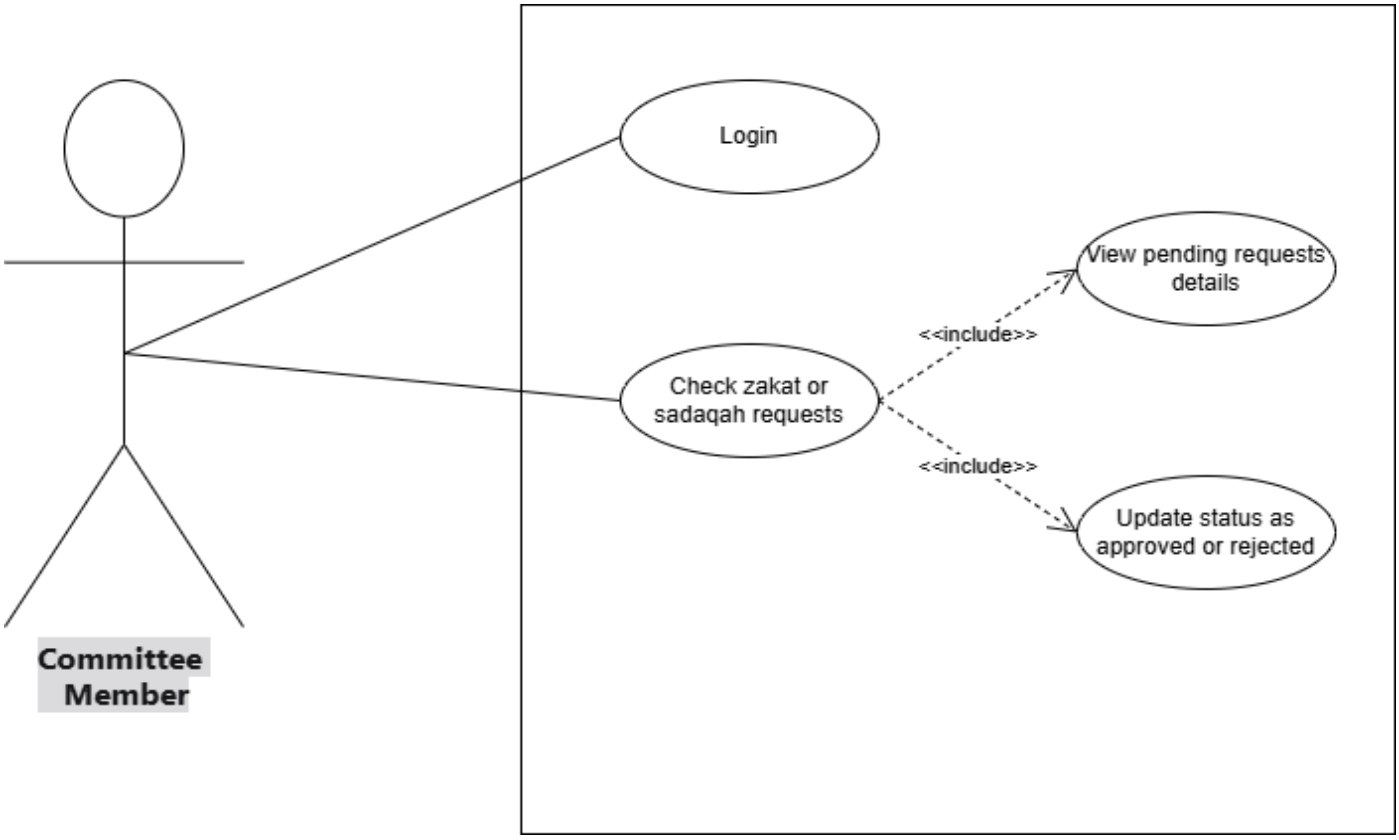

Figure 5 Use Case diagram of Committee Members

Storyboards

This section shows how users will use our system in real life. Each storyboard explains one main feature that happen on screen.

Storyboard 1 – Online Donation System

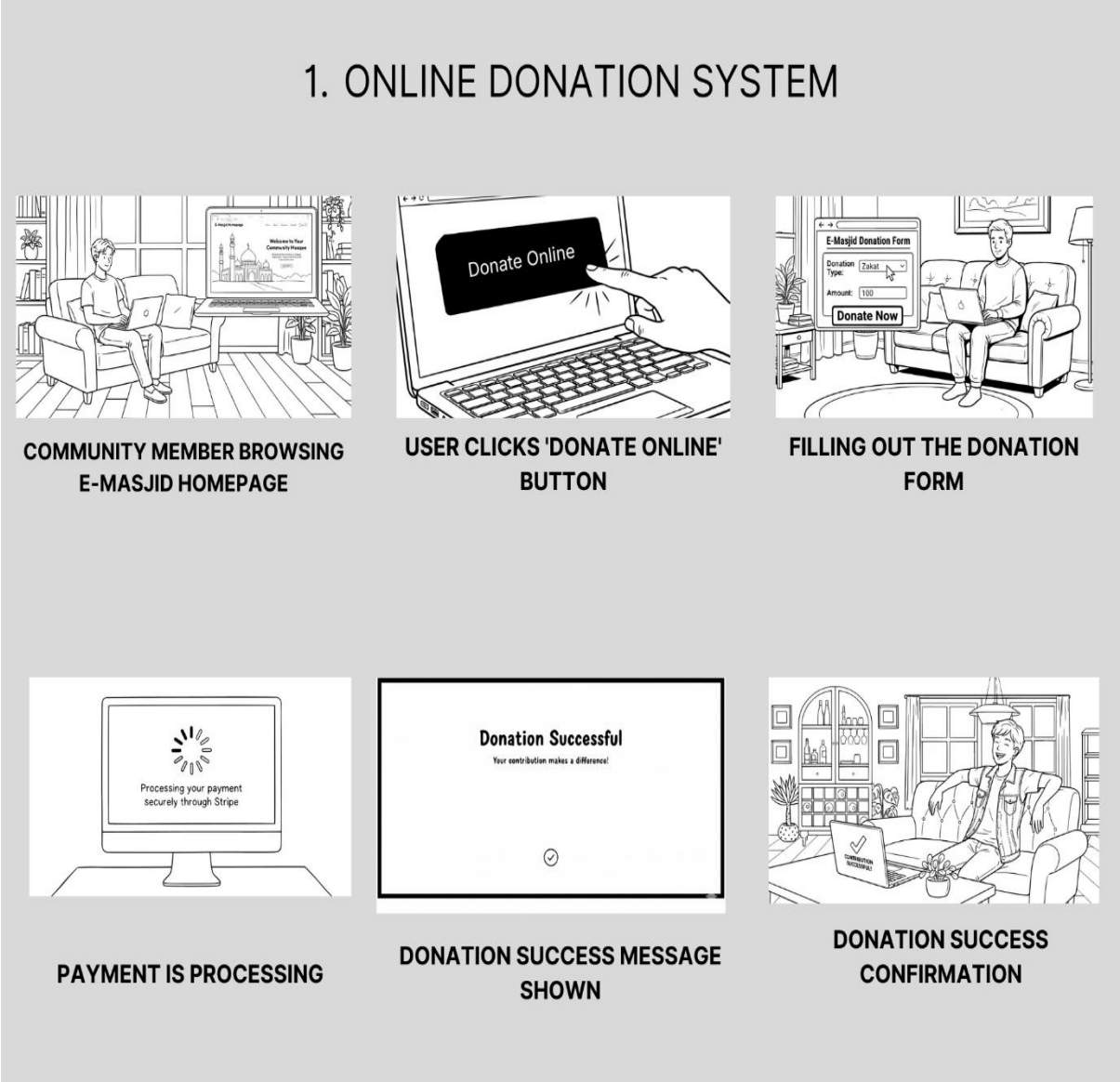

Figure 6 Online Donation Storyboard

Storyboard 2 – Event Management

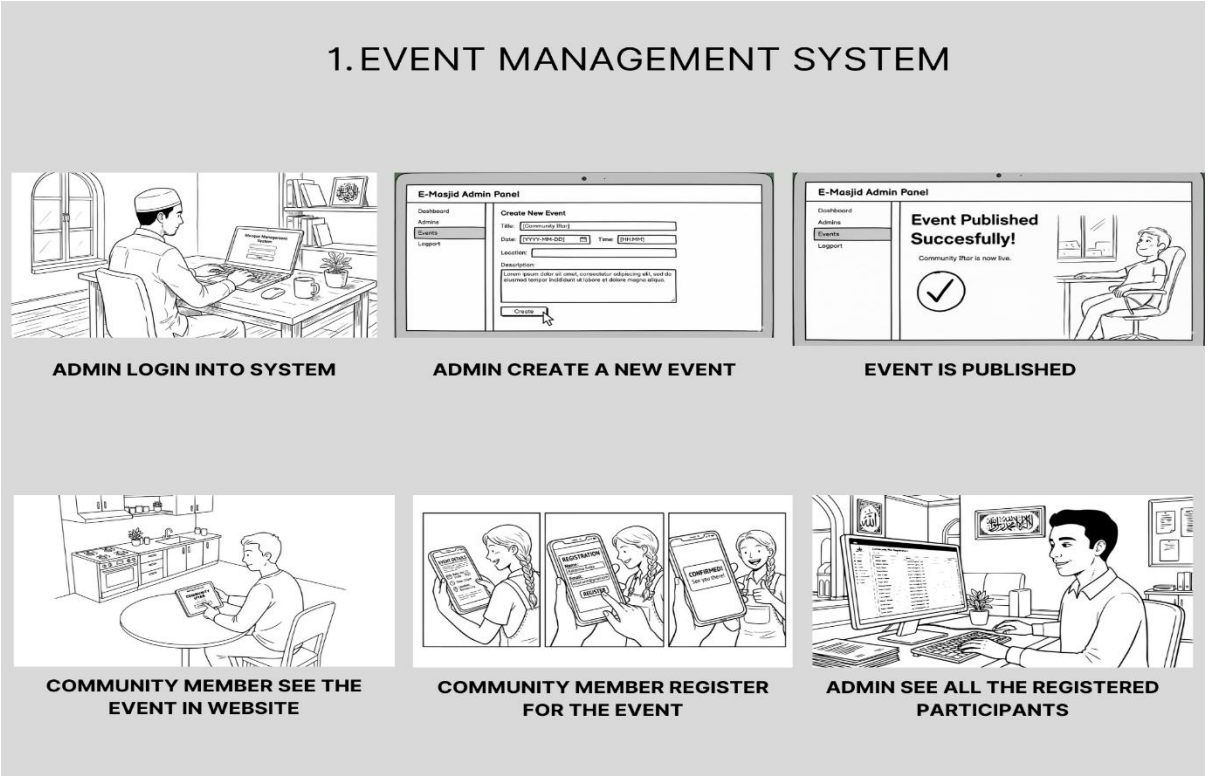

Figure 7 Event Management Storyboard

Storyboard 3 – Nikah Booking

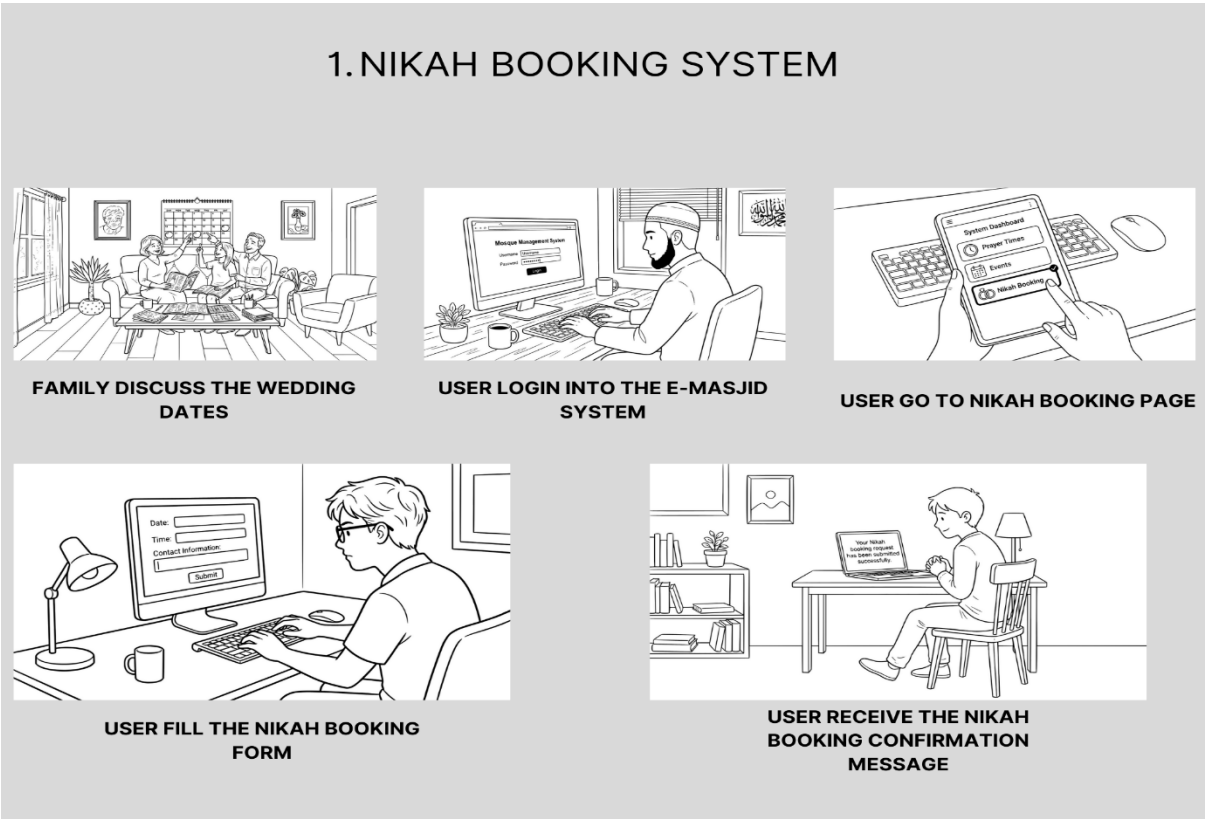

Figure 8 Nikah booking Storyboard

4.1. Summary

In this chapter, we described the detailed requirements of the E-Masjid System. We listed all six user classes and explained what each one can do. We wrote fourteen functional requirements covering everything. We also set clear non-functional requirements. These requirements give a proper understanding of what the system should do and how it should behave.

Chapter 4

Design and Architecture

3. System Design

Our E-Masjid System is a complete web based platform that will be accessible through any modern web browser. The system is designed to serve mosque administration for managing operations and community members can use these services.

Dependencies

1. Stable internet connection for all users.
2. Stripe service availability for payment processing.
3. Modern web browsers supporting React.js features.
4. MongoDB database server for data storage.

Interaction with Other Systems

1. **Stripe Payment Gateway:** It is used for making online donations securely.
2. **Email Service:** It is used for sending password reset links.
3. **Internet Connection:** It is required for all users to access the system.

Design Constraints

1. **Performance Requirements:** Prayer times page loads within 3 seconds, handles 100+ users during Friday prayers.
2. **Usability Requirements:** Simple interface with large buttons, works on mobile and computer, elderly friendly design.
3. **Security Requirements:** Encrypted passwords, secure payments through Stripe, admin access protection.
4. **Technical Constraints:** MERN stack technology, responsive design, automatic weekly backups.

4.1. Design considerations

Assumptions

Following are the assumptions:

1. Mosque administrators have basic computer knowledge.
2. Users have internet access and email accounts.

3. The mosque has at least one computer for admin use.
4. Religious scholars can use basic web applications.
5. Community members can use web browsers on phones or computers.
6. The Mosque Manager is a responsible person who knows which masjids to give access to.
7. Community members who submit zakat requests will give correct and honest information.
8. Committee members have email addresses and can check them regularly.

Dependencies

Following are the dependencies:

1. Stable internet connection for all users.
2. Stripe payment service available at any time.
3. Web browsers supporting modern JavaScript.
4. MongoDB database running properly.
5. Nodemailer library and a working email service for sending committee notifications and password reset emails.

Limitations

Following are the limitations:

1. Cannot work without internet connection.
2. No SMS notifications for announcements.
3. No mobile app version.
4. Payment system requires card payments only.
5. Cannot handle offline data entry.
6. The system does not automatically verify the identity of fund requesters and the committee must do that manually.

Risks

Following are the risks:

1. Payment security issues.
2. System downtime during prayer times.
3. Elderly users finding the system difficult.
4. Data loss from system crashes

4.2. Design Models

In this section, we show how we designed the E-Masjid System. We made different diagrams to explain the system structure, how data is organized, and how different parts of the system work together. These pictures make the design easy to understand for everyone.

Design Class Diagram

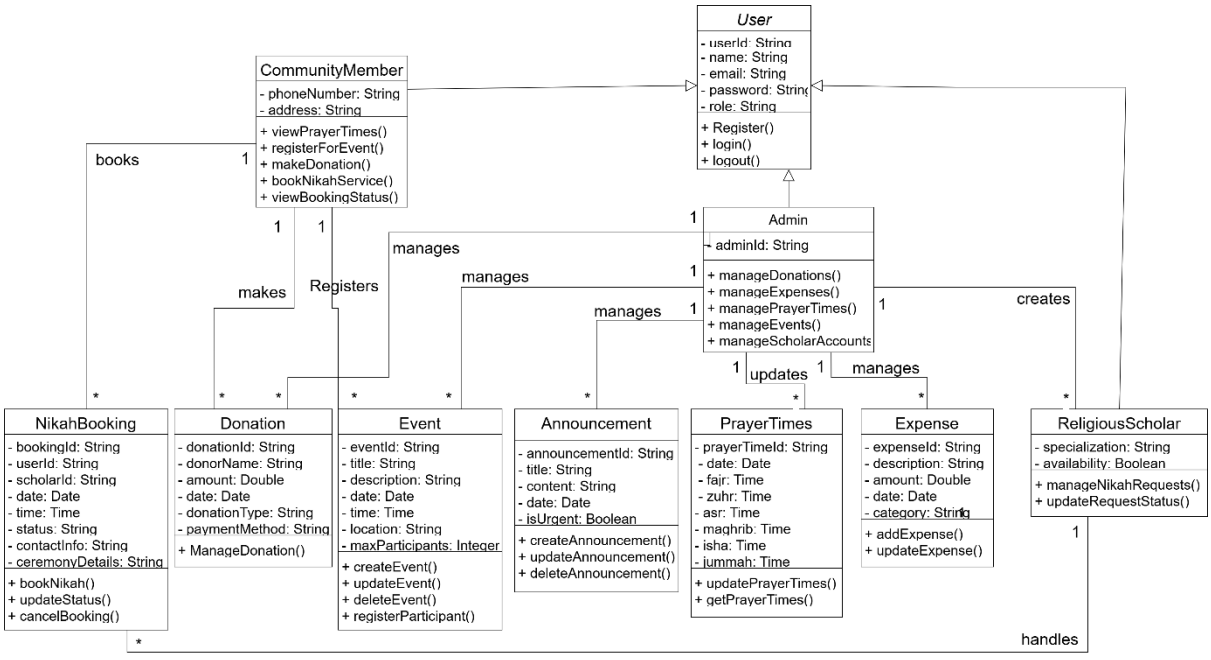

Figure 9 Class diagram

Sequence Diagram

User Login Sequence Diagram

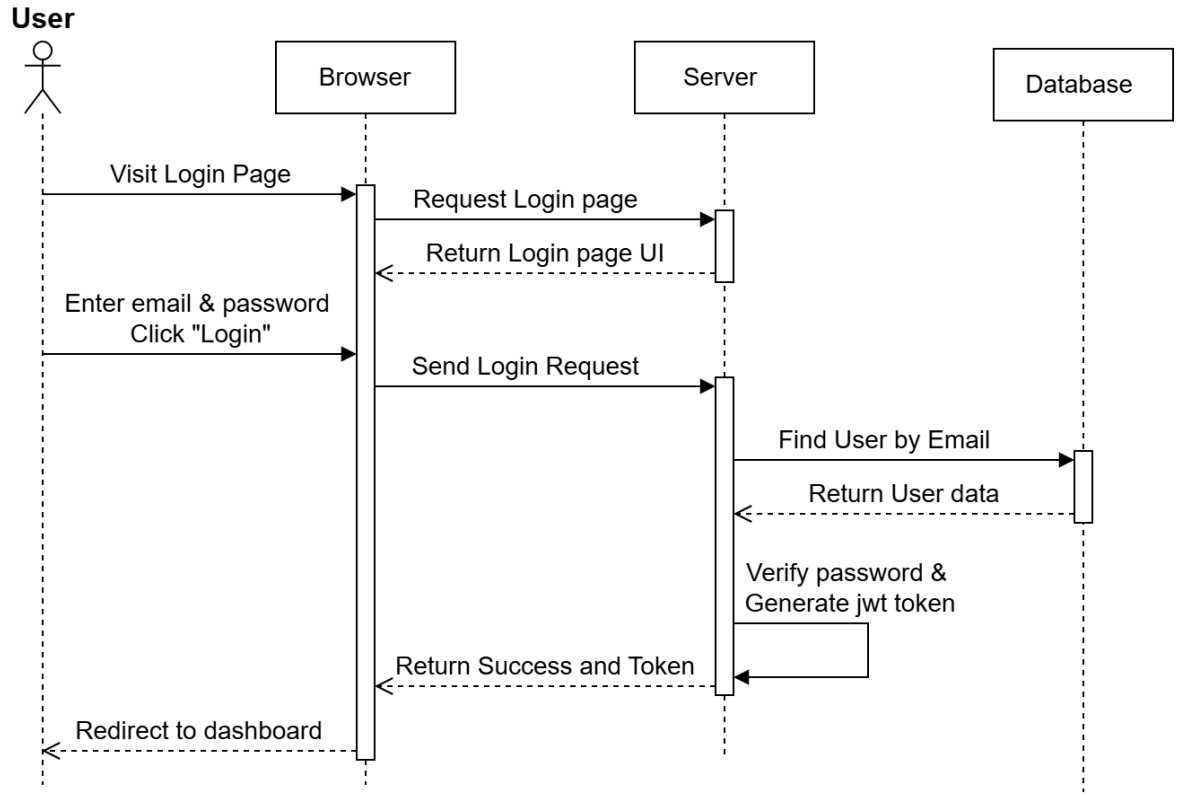

Figure 10 Login Sequence Diagram

Online Donation Sequence Diagram

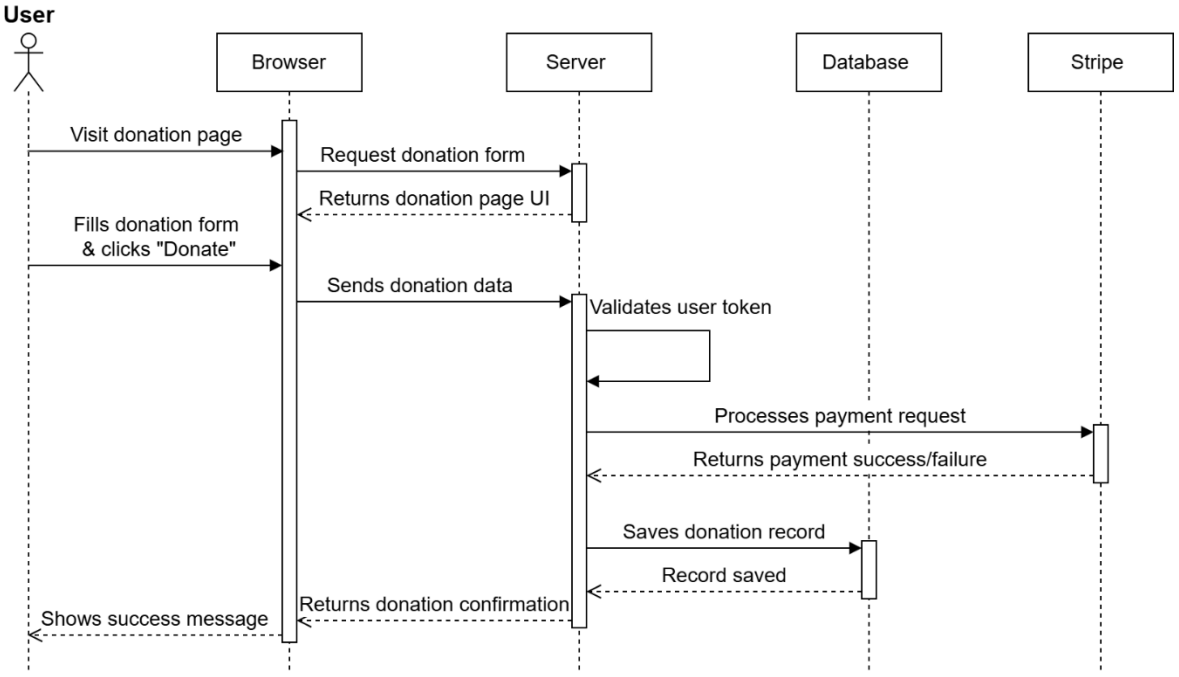

Figure 11 Online Donation Sequence Diagram

Nikah Booking Sequence Diagram

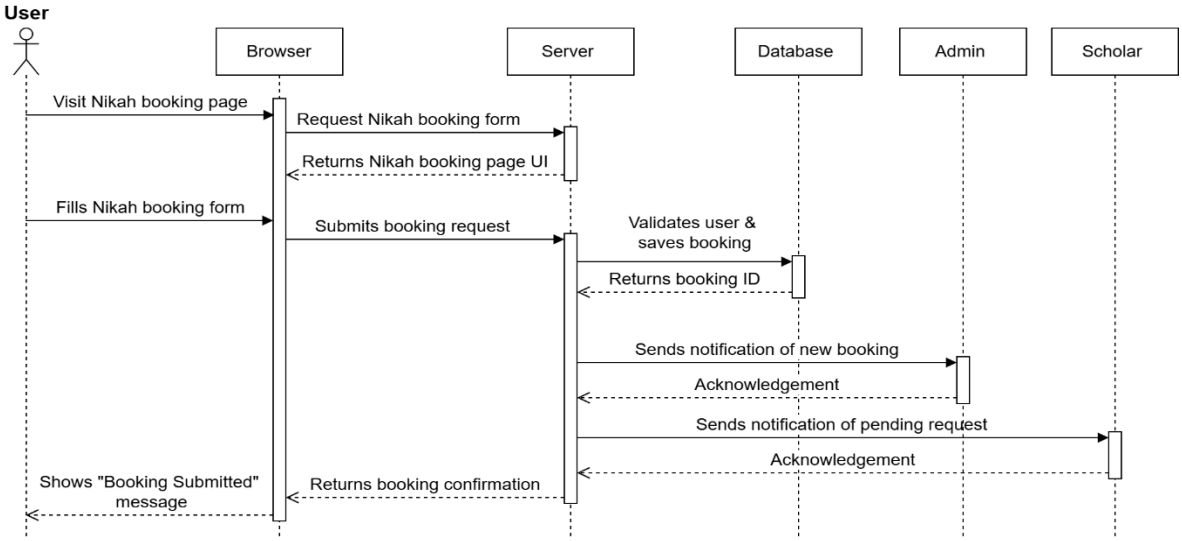

Figure 12 Nikah Booking Sequence Diagram

Prayer Times Update Sequence Diagram

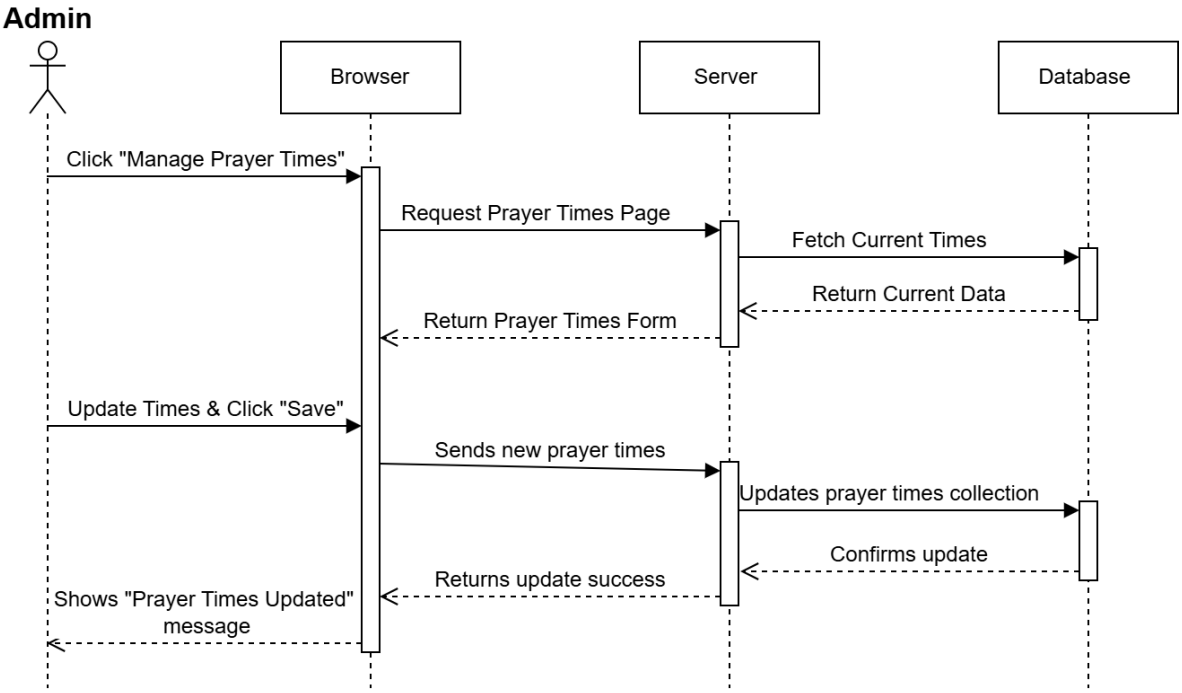

Figure 13 Admin Update Prayer Times Sequence Diagram

State Transition Diagram

These diagrams show how the system status changes when a user performs different actions.

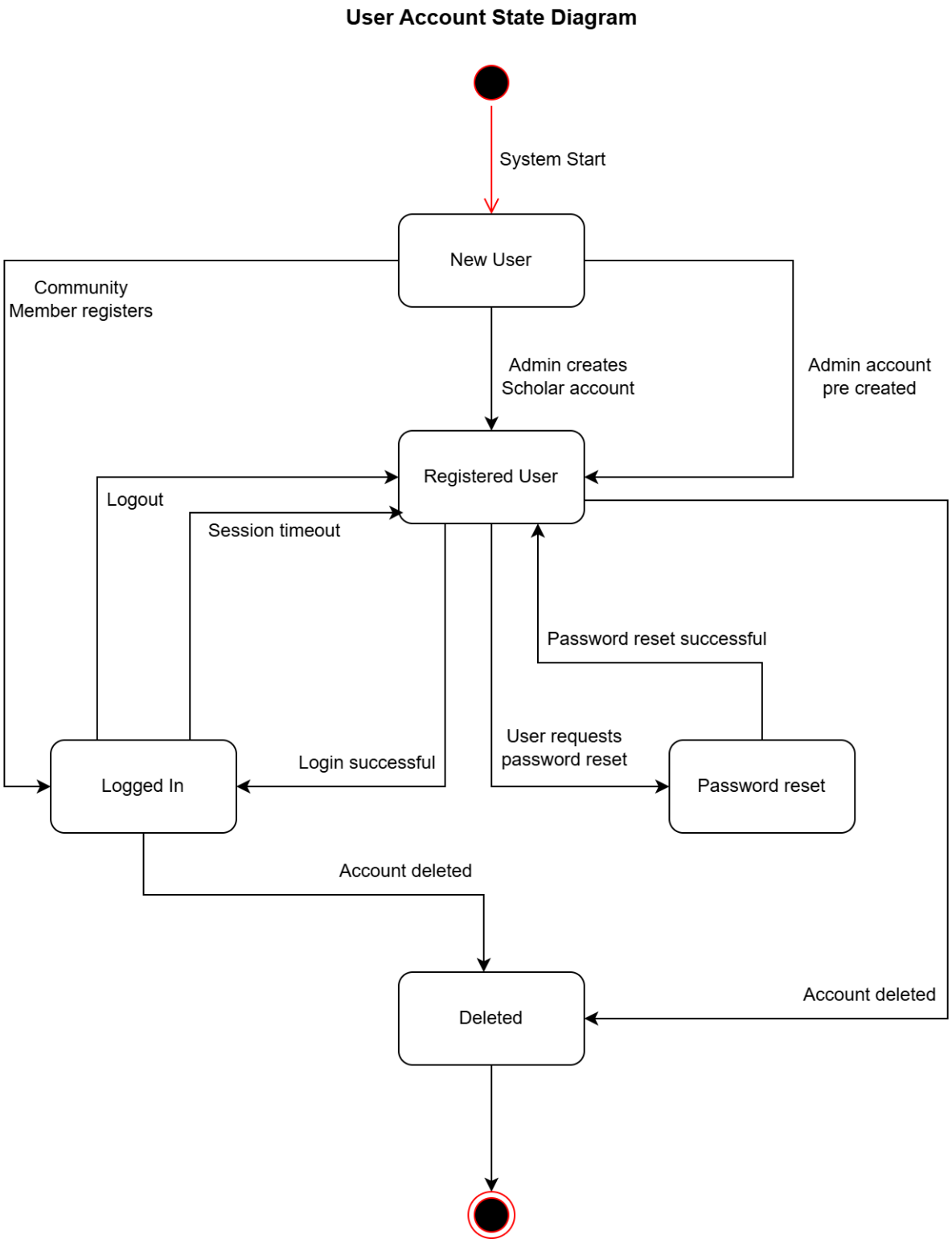

Figure 14 User Account State Diagram

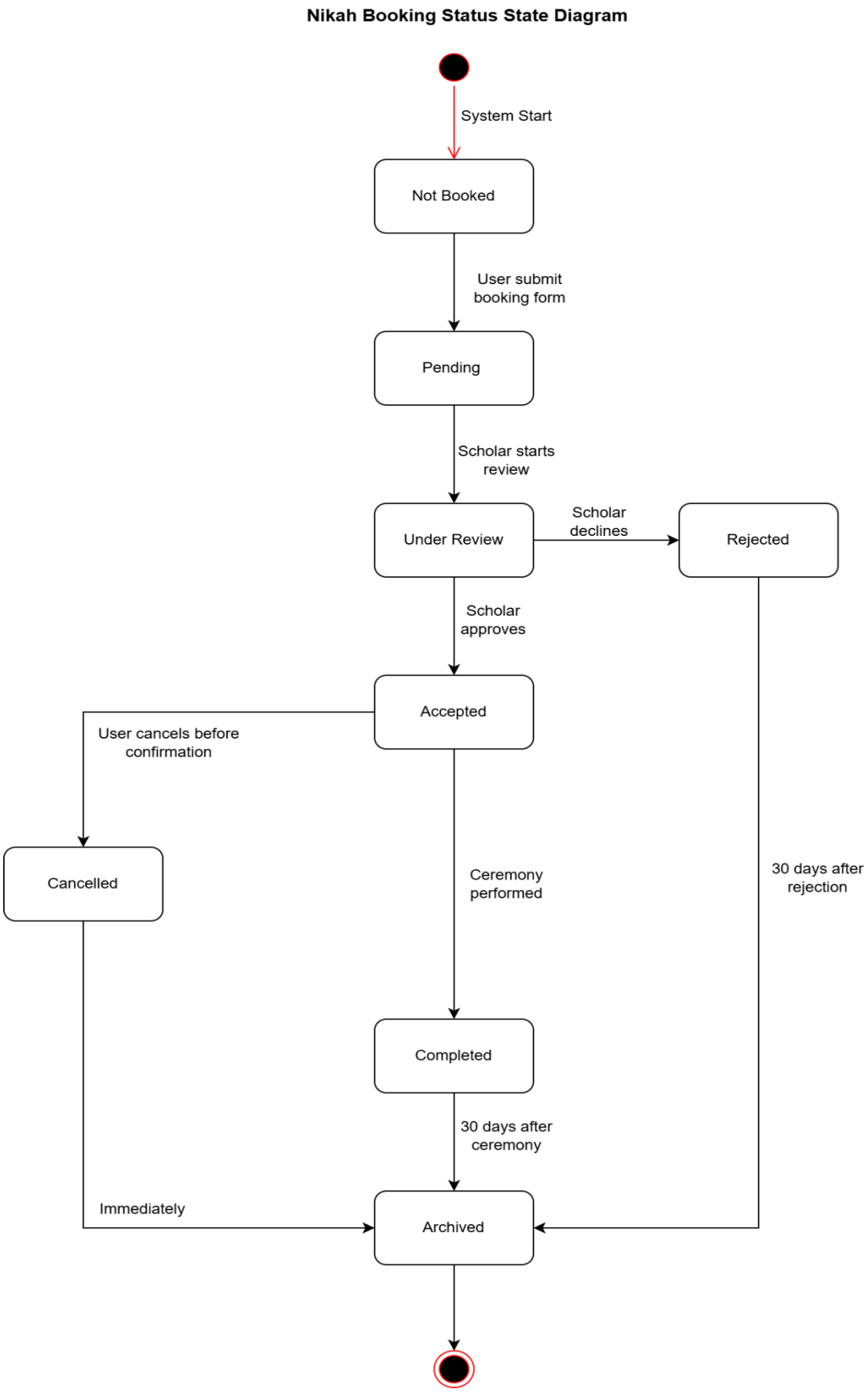

Figure 15 Nikah Booking Status State Diagram

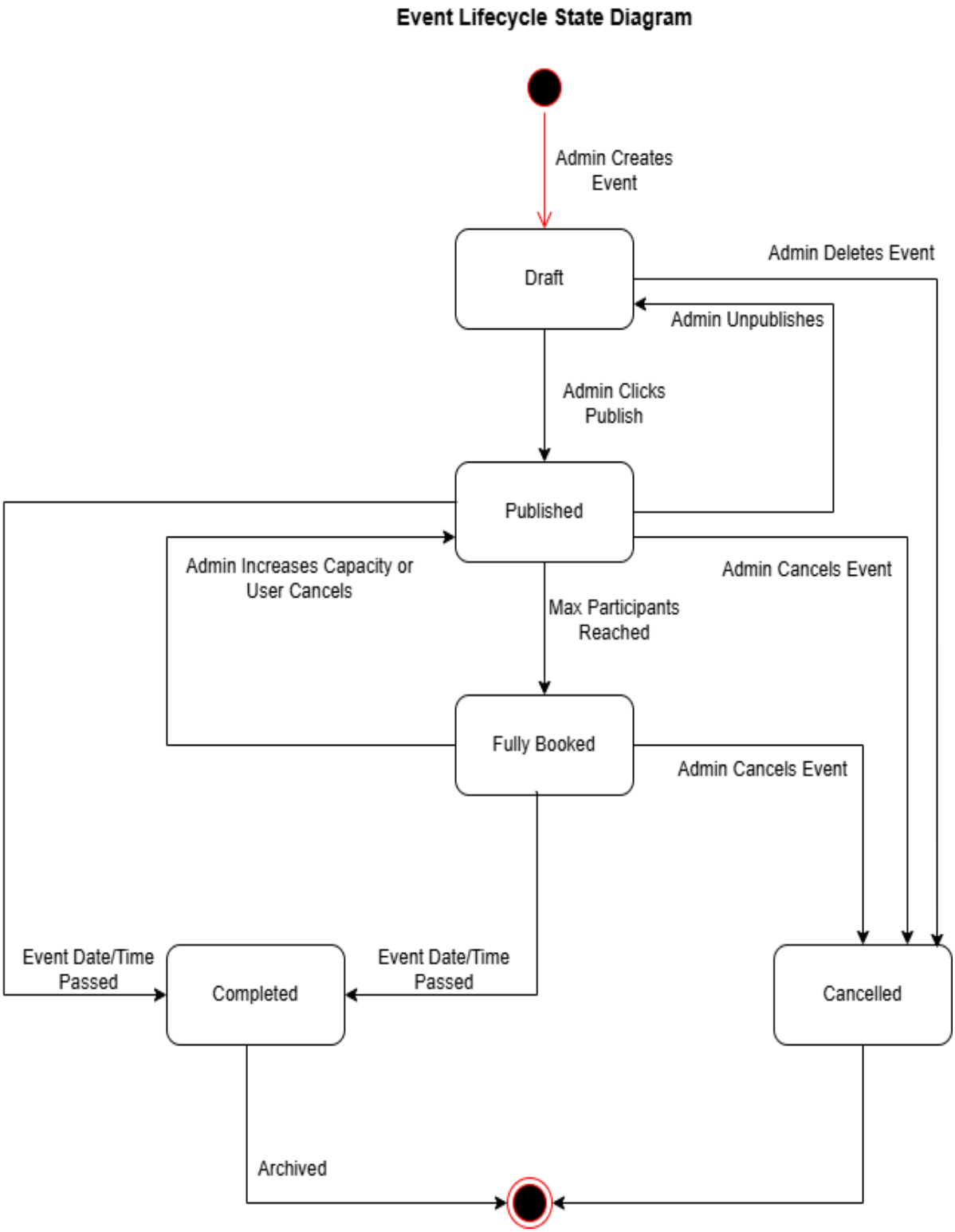

Figure 16 Event State Diagram

4.3. Architectural Design

Our E-Masjid System follows the MVC architecture pattern. This separates the system into three main parts.

1 **Model:** Handles data and business logic.

2 **View:** User interface that users see.

3 **Controller:** Processes user requests and connect Model and View.

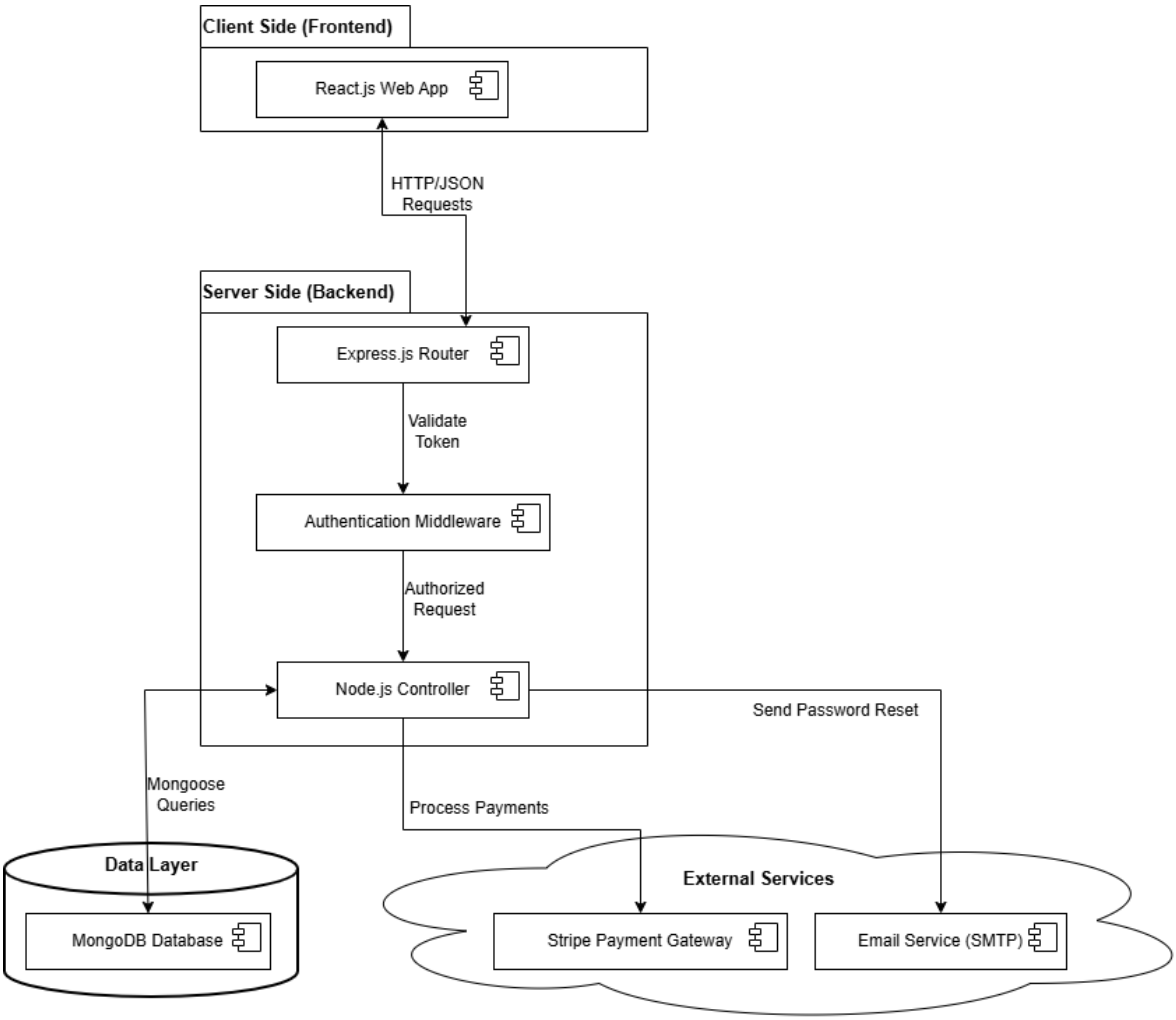

Figure 17 Architectural diagram

UML Component diagram

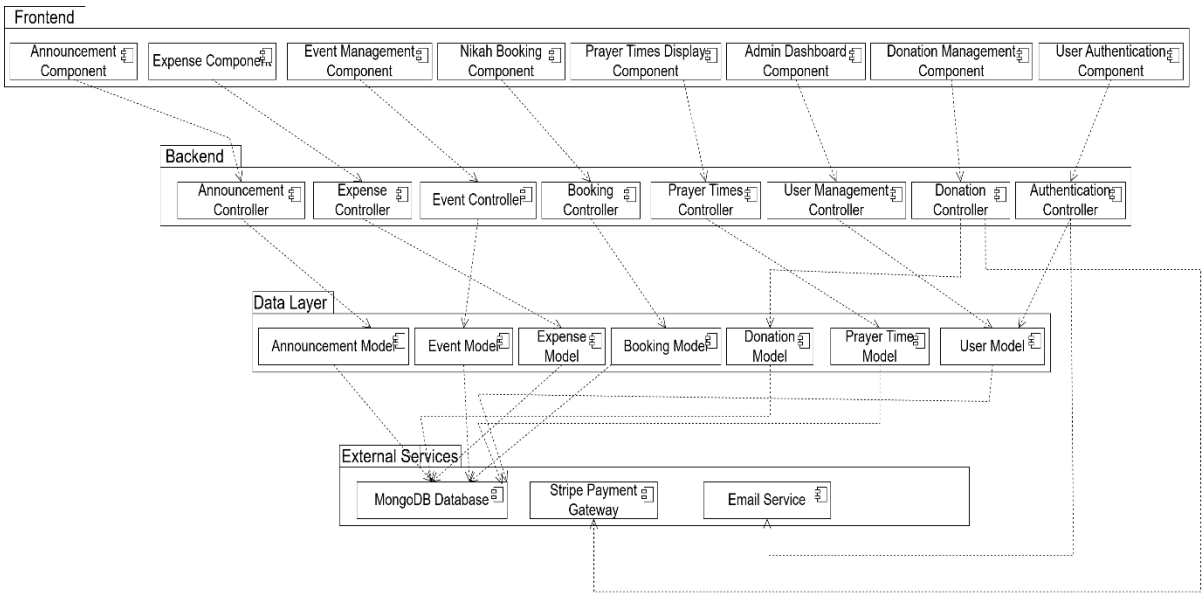

Figure 18 Component diagram

4.3.1 Data Design

Our system uses MongoDB database to store information. We have separate collections for different things like users, donations, events etc. Each collection keeps related information together. Admin and religious scholars are both in the users collection, with different role values.

Database Collections Structure:

1. **Users Collection**
   1.1. Stores all user information including admin, community members, and religious scholars.
   1.2. Uses role based access control.
   1.3. Fields: userId, name, email, password, role, phone, address, specialization.
2. **Donations Collection**
   2.1 Records all donation transactions.
   2.2 Links to donor information for transparency.
   2.3 Fields: donationId, donorId, amount, date, type , paymentMethod.
3. **Expenses Collection**
   3.1. Tracks mosque expenditure for financial transparency.
   3.2. Categorized expenses for better reporting.
   3.3. Fields: expenseId, description, amount, date, category.

4. **Events Collection**
   4.1. Manages mosque events and programs.
   4.2. Supports online registration.
   4.3. Fields: eventId, title, description, date, time, location, maxParticipants, registeredUsers[ ].
5. **Announcements Collection**
   5.1. Stores important mosque announcements.
   5.2. Supports urgent flag for important updates.
   5.3. Fields: announcementId, title, content, date, isUrgent, publishedBy.
6. **Prayer Times Collection**
   6.1. Stores daily prayer schedules.
   6.2. Special entries for Jummah and Ramadan.
   6.3. Fields: prayerTimeId, date, fajr, zuhr, asr, maghrib, isha, jummah, isSpecial.
7. **Nikah Bookings Collection**
   7.1.Manages nikah service requests
   7.2.Tracks booking status.
   7.3.Fields: bookingId, userId, scholarId, date, time, status, contactInfo, ceremonyDetails.
8. **Mosques Collection**
   8.1.Manages different mosques
   8.2.Fields: mosqueId, name, location, contactPerson, adminUserId, enabledModules , createdBy.
9. **ZakatRequests Collection**
   9.1.Manage zakat fund requests
   9.2.Fields: requestId, userId , mosqueId, requesterName, phone, reason, amountRequested, status, reviewedBy, reviewComment, createdAt, updatedAt.

Data Relationships:

1. One-to-many: One user can make multiple donations.
2. One-to-many: One admin can create multiple events.
3. Many-to-many: Many users can register for many events.
4. One-to-one: Each day has one prayer time schedule.

4.3.2 Data Dictionary

Table 41 Data Dictionary Table

Terminology	Description
Users Collection	Stores all system user accounts
userId	String, Primary key for user identification
Name	String, Full name of the user
Email	String, User email address
Password	String, Encrypted password for security
Role	String, User role (admin/community/scholar)
Phone	String, Contact phone number.
Address	String, User residential address
Specialization	String, For scholars area of expertise
Donations Collection	Records all financial donations
donationId	String, Unique donation identifier
donorId	String, Reference to user who donated
Amount	Number, Donation amount in rupees
Date	Date, When donation was made
Type	String, Donation type (Zakat/Sadaqah/Mosque Fund)
paymentMethod	String, Cash or Card payment
Expenses Collection	Tracks mosque spending
expenseId	String, Unique expense identifier
Description	String, Where the money was spent on

Amount	Number, Expense amount in rupees
Date	Date, When fund is spent
Category	String, Expense category
Events Collection	Manages mosque events
eventId	String, Unique event identifier
Title	String, Event name/title
Description	String, Detailed event information
Date	Date, Event date
Time	Time, Event time
Location	String, Where event will be held
maxParticipants	Number, Maximum allowed attendees
registeredUsers	Array, List of user IDs who registered
Announcements Collection	Stores mosque announcements
announcementId	String, Unique announcement identifier
Title	String, Announcement headline
Content	String, Full announcement text
Date	Date, When announcement was posted
isUrgent	Boolean, Marks urgent announcements
publishedBy	String, Admin who posted the announcement
PrayerTimes Collection	Stores prayer schedules
prayerTimeId	String, Unique prayer time identifier
Date	Date, Date for prayer times

Fajr	Time, Fajr prayer time
Zuhr	Time, Zuhr prayer time
Asr	Time, Asr prayer time
Maghrib	Time, Maghrib prayer time
Isha	Time, Isha prayer time
Jummah	Time, Jummah prayer time
NikahBookings Collection	Manages marriage service requests
bookingId	String, Unique booking identifier
userId	String, Reference to user who booked
scholarId	String, Reference to assigned scholar
Date	Date, Requested ceremony date
Time	Time, Requested ceremony time
Status	String, Current status (Pending/Accepted/Rejected)
contactInfo	String, User contact details for ceremony
ceremonyDetails	String, Additional ceremony information
Mosques Collection	Stores details of each mosque registered in the system.
mosqueId	String, unique identifier for the mosque.
name	String, store name of mosque
Location	String, store address of mosque
contactPerson,	Number, store contact information
adminUserId	Number, store unique admin id
enabledModules	Array of strings, Decides which features are shown for that mosque.

createdBy	Date, store created date of mosque
ZakatRequests Collection	Stores all Zakat and Sadaqah help requests.
requestId	String, unique request identifier.
userId	String, reference to the user who submitted the request.
status	String, one of: Pending, Approved, Rejected.
reviewedBy	String, user ID of the committee member who took the decision.
reviewComment	String, reason given by the committee.

5.

4.4. User Interface Design

The E-Masjid System will have a clean, simple, and easy to use interface designed for all types of users, including elderly people who may not be comfortable with complex technology.

User Experience

1. **Homepage:** Shows current prayer times, recent announcements, and quick access buttons for main features.
2. **Navigation:** Simple menu at the top with clear labels.
3. **Mobile Friendly:** All screens work perfectly on mobile phones and tablets.
4. **Elderly Friendly:** Large buttons, clear text, and simple forms.

How Different Users Will Use the System:

For Community Members:

1 Prayer times always visible on top of home page.

2 Simple donation form with card payment option.

3 Registration button for events.

4 Easy booking form for nikah.

5 View booking status in mybooking page.

For Mosque Admin:

1 Special admin panel accessible through /admin URL.
2 Simple forms to add/update donations, expenses, events, announcements.
3 Financial reports showing income and expenses.
4 Create and manage religious scholar accounts.

For Religious Scholars:

1 View pending requests in simple list format.
2 One click buttons to accept or reject bookings.
3 See their booked ceremonies in calendar view.

Feedback and Messages:

1 Green popup messages for successful actions.
2 Red popup messages with simple explanations.
3 "Are you sure?" prompts for important actions.

4.4.1 Screen Images

We have created basic screen designs to show how the interface will look. These designs follow our guidelines of simplicity and ease of use.

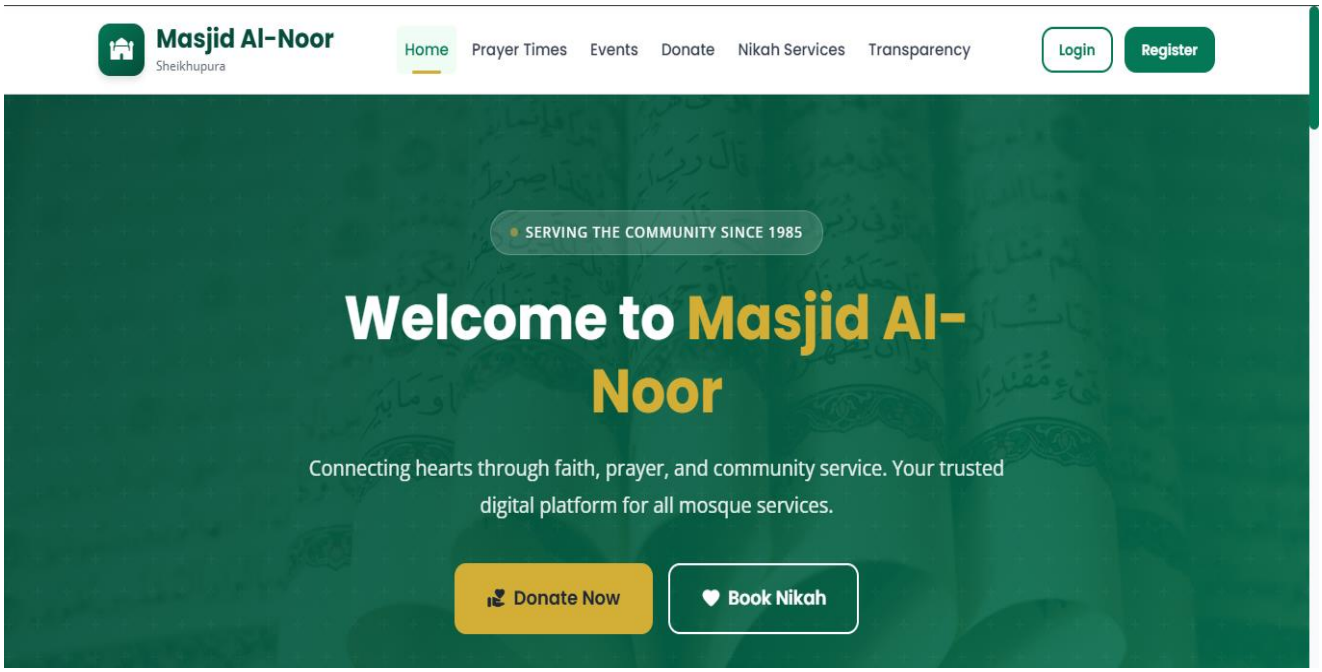

Figure 19 Home page design

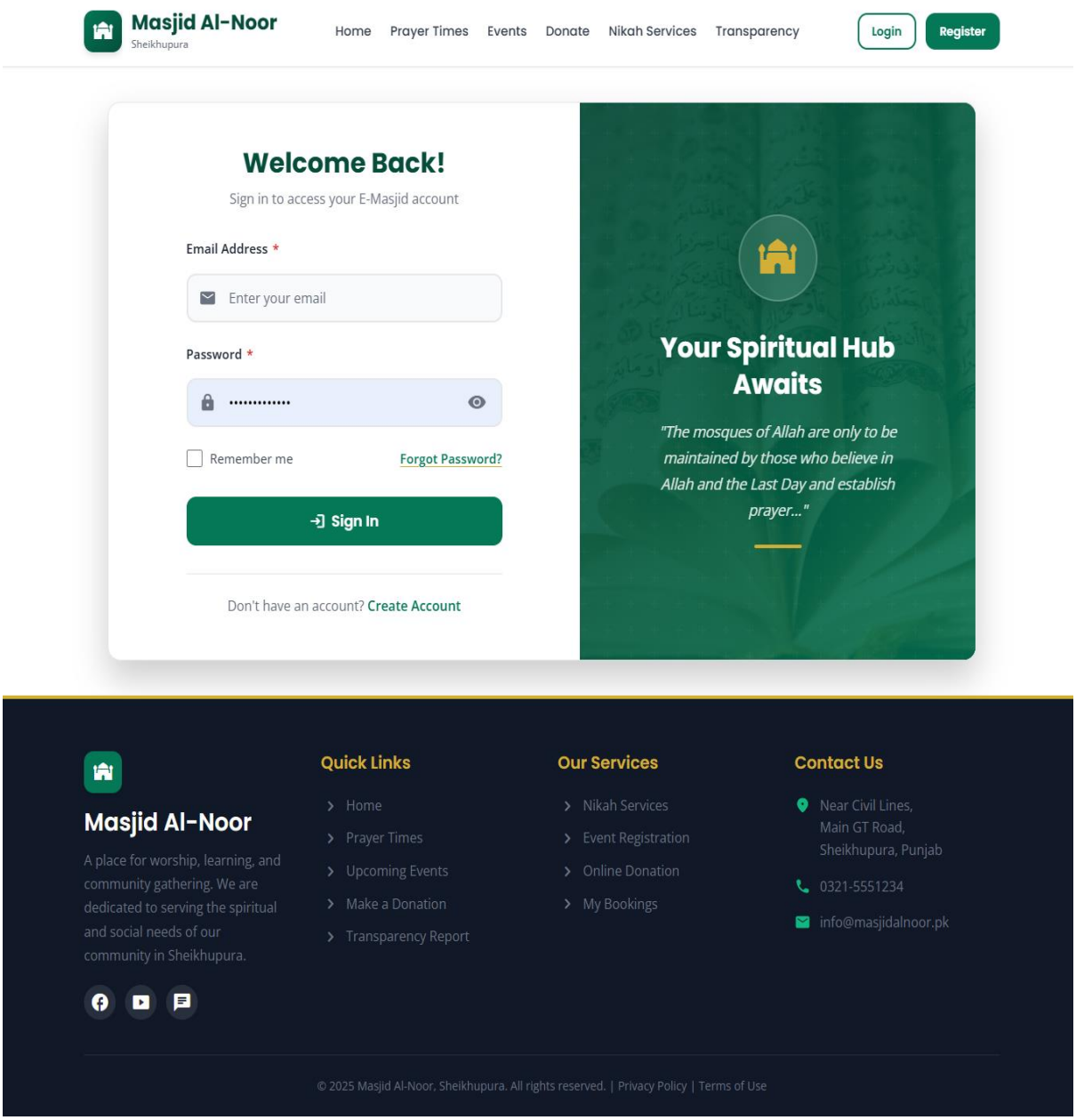

Figure 20 Login page design

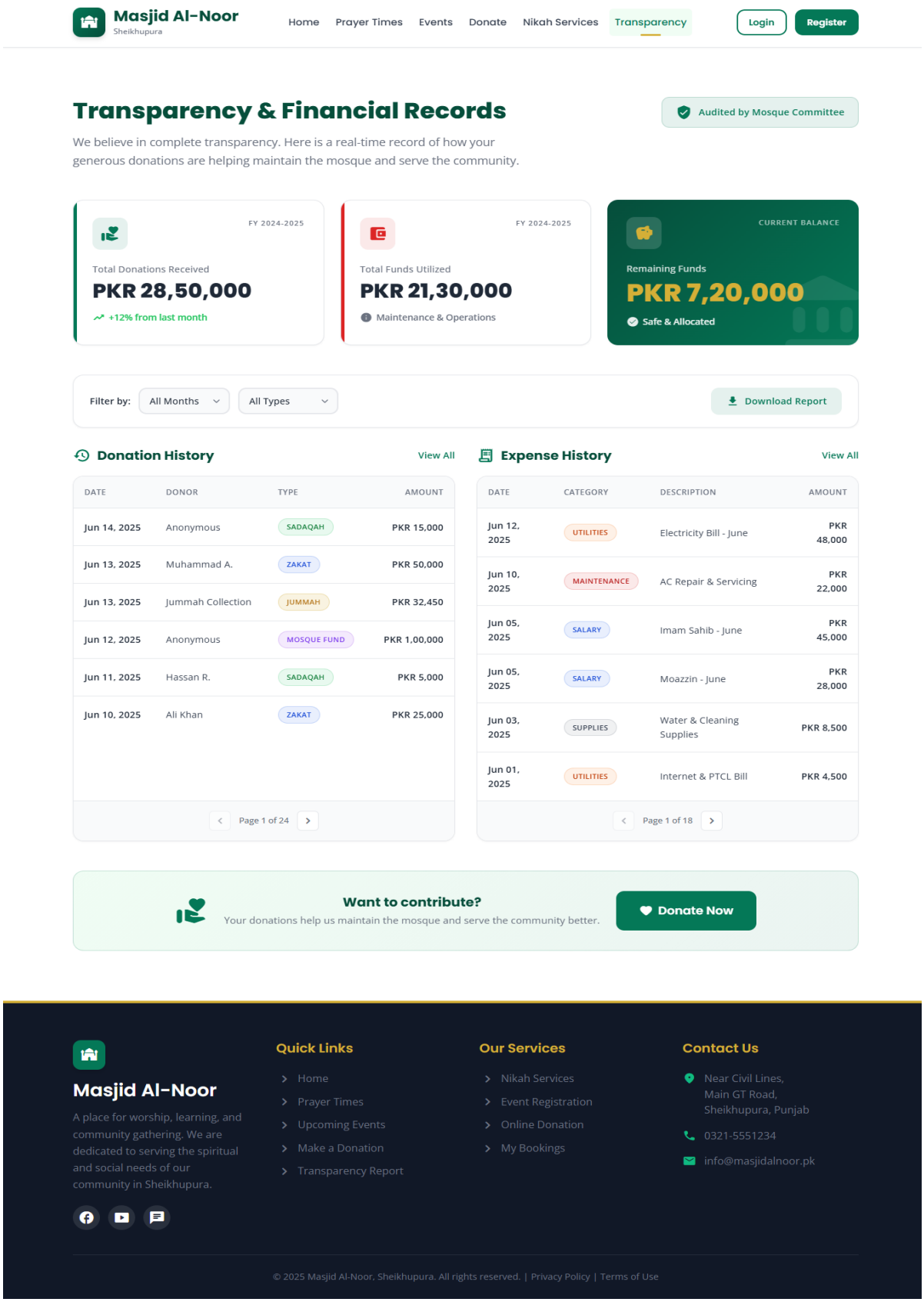

Figure 21 Donation Transparency page design

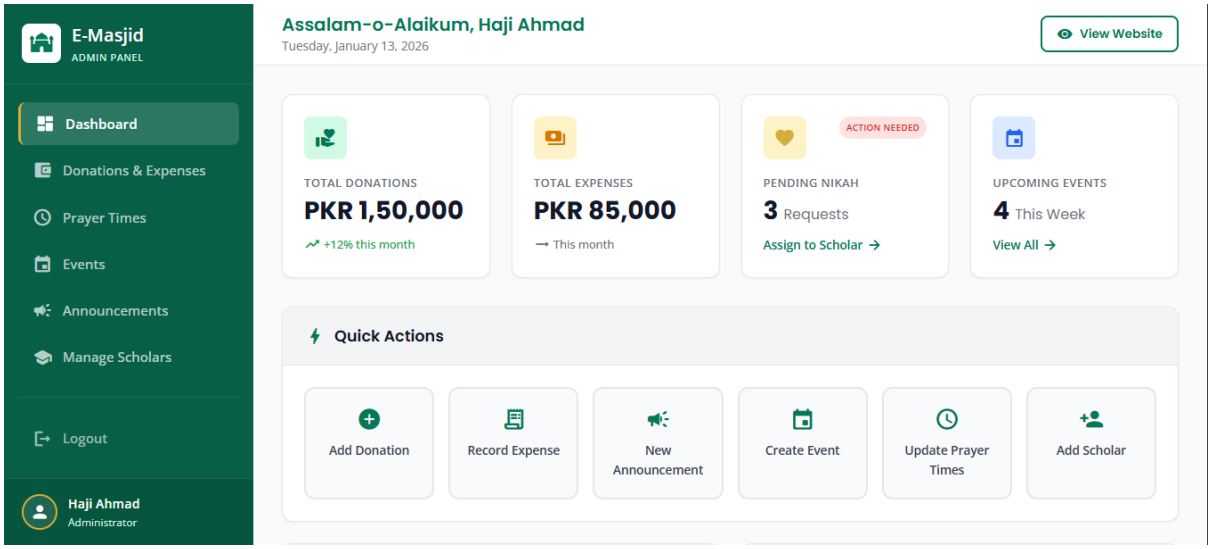

Figure 22 Admin Dashboard design

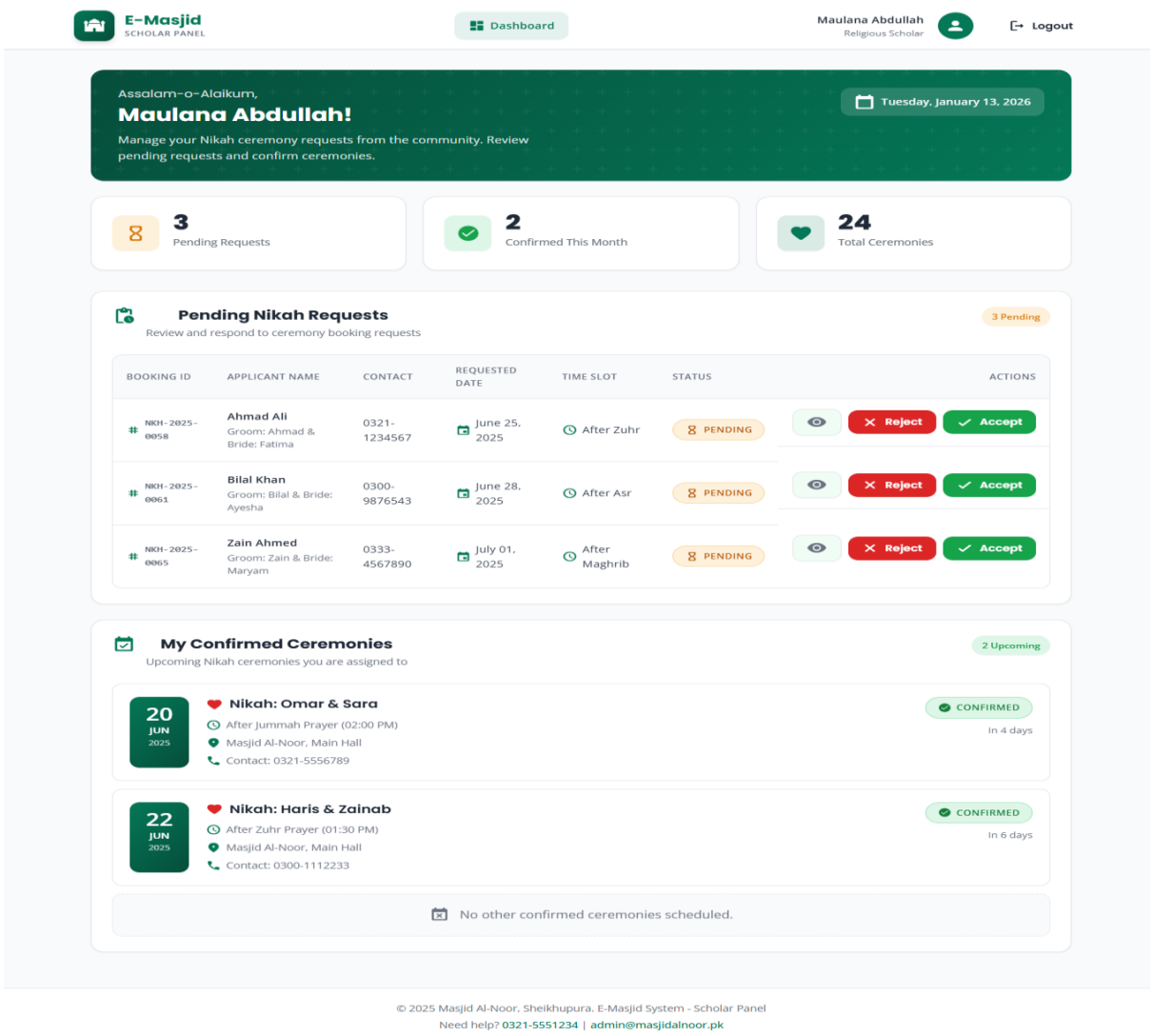

Figure 23 Scholar page design

4.4.2 Screen Objects and Actions

Use Case 1: Making an Online Donation

Screen Objects:

1. **Donation Amount Field**: Text box to enter donation amount.
2. **Donation Type**: Select Zakat, Sadaqah, or Mosque Fund.
3. **Card Details Form**: Fields for card number, expiry, CVC.
4. **Donate Now Button**: Green button to submit donation.

Actions:

1. **User enters amount**: System validates it is a positive number.
2. **User selects donation type**: System shows description of that type.
3. **User enters card details**: System validates card format.
4. **User clicks Donate Now**: System processes payment via Stripe.
5. **Payment successful**: Shows green "Donation Successful" message on the dashboard.
6. **Payment failed**: Shows red "Payment Failed" message with retry option.

Use Case 2: Admin Creating an Event

Screen Objects:

1 **Event Title Field**: Text box for event name.

2 **Date and Time Pickers**: Calendar and time selectors.

3 **Description Box**: Large text area for event details.

4 **Location Field**: Text box for event location.

5 **Max Participants Field**: Number field for attendance limit.

6 **Publish Button**: Blue button to publish event.

7 **Save Draft Button**: Gray button to save for later.

Actions:

1 **Admin enters event details**: System validates all required fields.

2 **Admin sets date/time**: System checks date is not in past.

3 **Admin sets max participants**: System validates positive number.

4 **Admin clicks Publish**: System creates event and shows on website.

5 **Event published**: Shows green "Event Published Successfully".

6 **Validation error**: Shows red message next to incorrect field.

Use Case 3: Submitting a Zakat Request

Screen Objects:

1. **Name Field**: Text box for requester name
2. **Phone Field**: Phone no of the person
3. **Reason Field**: large area text
4. **Amount Needed Field**: Number box for amount
5. **Submit Button**: submit the request

Actions:

1. **User enters details**: System checks that all fields are filled.
2. **User clicks Submit**: System saves request, sends emails to committee and shows a "Request Submitted" message.
3. **If any field is empty, a red message appears:** "Please fill all fields."

4.5 Design Decisions

This section explains the main design choices we made for the E-Masjid System and why we chose them.

We decided to use the MVC pattern for our system. This means we separate our code into three main parts which is model, view and controller

We are using the MERN stack for our project. We chose this because all parts use JavaScript which makes development faster.

We picked MongoDB instead of traditional SQL databases because it works naturally with JavaScript and Node.js.

We choose Stripe for online donations because it is very secure and handles card details safely and many other projects use it successfully.

We plan to deploy the system on cloud services because it is more reliable than our own computers and can handle more users during busy times. It automatically backs up data and it is affordable for a mosque budget.

4.6 Summary

This Software Design Specification show the complete technical plan for our E-Masjid System. We have explained the MVC architecture, MERN technology, database design, and user interfaces. The design describes all functional requirements and provides clear guidance for development.

Chapter 5

Implementation

5. Implementation

8.1. Algorithm

*Table 42 **Record Donation***

Algorithm 1 Record Donation
Input: donorId, amount, type, isAnonymous, paymentMethod
Output: donation record saved or error
1: Validate amount > 0, type not empty 2: If validation fails, return error "Invalid input" 3: Create donation object: donationId = generate unique id donorId = donorId amount = amount type = type paymentMethod = paymentMethod isAnonymous = isAnonymous date = current date 4: Save donation in Donations collection 5: If save fails, return error "Could not save donation" 6: Return success message "Donation recorded"

*Table 43 **Submit Zakat Request***

Algorithm 1 Submit Zakat Request
Input: userId, name, phone, reason, amount, mosqueId
Output: request saved, emails sent
1: Check all fields are filled 2: If any field empty, return error "Please fill all fields" 3: Create request object: requestId = generate unique id userId = userId mosqueId = mosqueId requesterName = name phone = phone reason = reason amountRequested = amount status = "Pending" createdAt = current date 4: Save request in ZakatRequests collection 5: If save fails, return error "Could not submit request" 6: Find all users with role="committee" and same mosqueId 7: For each committee member, get their email 8: Use Nodemailer to send email with request details to each email 9: Return success message "Request submitted. Committee will review."

Table 44 Committee Review Decision

Algorithm 1 Committee Review Decision
Input: requestId, committeeUserId, decision ("Approved" or "Rejected"), comment
Output: request status updated
1: Find request by requestId in ZakatRequests collection 2: If request not found, return error "Request does not exist" 3: **I**f request.status is not "Pending", return error "Request already reviewed" 4: Update request: status = decision reviewedBy = committeeUserId reviewComment = comment updatedAt = current date 5: Save updated request 6: If save fails, return error "Could not update request" 7: Return success message "Decision saved. Requester can now see the status."

8.2. External APIs/SDKs

Table 45 Details of APIs used in the project

Name of API and version	**Description of API**	**Purpose of usage**	**List down the API endpoint/function/class in which it is used**
Stripe API	Online payment processing	Secure online donations via card	stripe.paymentIntents.create()
Nodemailer	Email sending library for Node.js	Send password reset emails and notify committee of new Zakat requests	transporter.sendMail()
JSON Web Token	Token based authentication	User login and role based access	jwt.sign(), jwt.verify()
Mongoose	MongoDB object modeling	Connect and query MongoDB	mongoose.connect(), Model.find(), Model.save()

8.3. Code Repository

In order to manage the version control and collaboration effectively for the group project, Git will be used as the primary tool. All project files, including code, documentation, and other relevant resources, will be stored in the Git repository. This will ensure proper tracking of changes, collaboration among team members, and access to the most up-to-date version of the project.

Git Repository Link:
The repository for the group project can be accessed using the following link:
[https://github.com/dawood125/E-Masjid-System]

5.3.1 Metrics of the Git Repository:

The following metrics will be monitored to ensure effective use of the Git repository:

Commits: The number of commits made to track the frequency and consistency of contributions by team members.

Branches: The number of active and merged branches, representing the organization of different features and stages of development.

Pull Requests: The number of pull requests created, merged, or in review, showing the collaboration and code review process.

Issues: The number of open and closed issues, indicating bug tracking, feature requests, or task management.

Contributors: A list of contributors and their contribution statistics (commits, lines added, and lines removed).

Code Reviews: The number of reviews conducted on pull requests to ensure code quality and compliance with project standards.

8.4. Summary

In this chapter, we described the main algorithms used in the system. We listed the external APIs that the project depends on. The code is managed through a Git repository with proper branching and pull request practices. All major features have been implemented according to the design and the system is ready for testing and deployment.

Chapter 6

Testing and Evaluation

6. Introduction

Testing is important to make sure our system works correctly and does what we promised in the requirements. We tested the E-Masjid System in four ways: unit testing for small parts, functional testing for full features, integration testing to check how parts work together, and performance testing for speed and stability. Below are the test cases we ran. We followed the testing best practices given in the template.

8.1. Unit Testing (UT)

We tested individual backend functions to make sure they work properly on their own.

Table 46 Testcase User Registration

Field	Value
Testcase ID	UT-1
Requirement ID	FR-1
Title	User Registration
Description	Check if a new user can register with correct details
Objective	Make sure the registration function saves the user and returns success
Driver/precondition	Server running, database empty of this user
Test steps	1. Send a POST request to /api/auth/register with name, email, password, role 2. Check the response 3. Check the database
Input	name: "Ahmed", email: "ahmed@test.com", password: "123456", role: "community"
Expected Results	Status 200, message "Registration successful", user found in database
Actual Result	Status 200, message "Registration successful", user saved in Users collection
Remarks	Pass

Table 47 Testcase Anonymous Donation Recording

Field	Value
Testcase ID	UT-2

Requirement ID	FR-2
Title	Record Donation with Anonymous
Description	Test if a donation marked anonymous saves correctly
Objective	Make sure the isAnonymous field is stored as true
Driver/precondition	User logged in, database connected
Test steps	1. Call the donation recording function with isAnonymous = true 2. Check the saved document
Input	donorId: "...", amount: 500, type: "Sadaqah", isAnonymous: true
Expected Results	Donation saved, isAnonymous field is true
Actual Result	Donation saved with isAnonymous: true
Remarks	Pass

Table 48 Testcase Zakat Request Validation

Field	**Value**
Testcase ID	UT-3
Requirement ID	FR-13
Title	Zakat Request Empty Field Validation
Description	Check if the system rejects a request with missing fields
Objective	Make sure validation works before saving
Driver/precondition	User logged in
Test steps	1. Send POST to /api/zakat-request with reason field empty
Input	name: "Ali", phone: "0300", reason: "", amount: 1000
Expected Results	Error message "Please fill all fields"
Actual Result	Error message "Please fill all fields"
Remarks	Pass

8.2. Functional Testing (FT)

We tested each module from the user's point of view, making sure the whole feature works.

Table 49 Testcase Community Member Makes Online Donation

Field	Value
Testcase ID	FT-1
Requirement ID	FR-8
Title	Online Donation using Stripe
Description	A logged in community member donates online with card
Objective	Check that the payment flow works and donation is recorded
Driver/precondition	User logged in, Stripe test mode active
Test steps	1. Click "Donate Online" 2. Select type and enter amount 3. Enter test card details (4242...) 4. Click "Donate Now"
Input	Amount: 1000, type: Mosque Fund, card: 4242 4242 4242 4242
Expected Results	Green message "Donation Successful", donation appears in records
Actual Result	Donation Successful message, record added in database
Remarks	Pass

Table 50 Testcase Zakat Request Submission

Field	Value
Testcase ID	FT-2
Requirement ID	FR-13
Title	Submit Zakat Request
Description	A community member submits a request for Zakat help
Objective	Check that request is saved and emails are sent to committee
Driver/precondition	User logged in, at least one committee member with email exists
Test steps	1. Click "Request Zakat Help" 2. Fill all fields 3. Click Submit
Input	Name, Phone, Reason: "Need help for medical", Amount: 5000
Expected Results	Request saved with status Pending, email sent to committee
Actual Result	Request saved, email received in committee inbox

Remarks	Pass

Table 51 Testcase Committee Approves Request

Field	Value
Testcase ID	FT-3
Requirement ID	FR-14
Title	Committee Approves Zakat Request
Description	Committee member reviews and approves a pending request
Objective	Check that status changes and requester can see it
Driver/precondition	Committee member logged in, request exists
Test steps	1. Open review dashboard 2. Click on a pending request 3. Click "Approve" and write comment
Input	Comment: "Verified, deserving case"
Expected Results	Status changed to Approved, requester sees "Approved" on their page
Actual Result	Status Approved, requester view shows approved
Remarks	Pass

Table 52 Testcase Mosque Manager Creates Mosque

Field	Value
Testcase ID	FT-4
Requirement ID	FR-12
Title	Mosque Manager Adds a New Mosque
Description	Mosque Manager creates a mosque with selected modules
Objective	Check that mosque is saved and admin account is created
Driver/precondition	Mosque Manager logged in
Test steps	1. Go to "Manage Mosques" 2. Click "Add New Mosque" 3. Fill name, location, select modules 4. Click Save
Input	Name: "Madina Masjid", Location: "Sheikhupura", Modules: donations, events, prayer

Expected Results	Mosque created, admin account linked, only selected modules visible
Actual Result	Mosque saved, admin can log in and see only selected features
Remarks	Pass

Table 53 Testcase Promotional Homepage Section

Field	Value
Testcase ID	FT-5
Requirement ID	FR-5
Title	Promotional Section on Homepage
Description	Check that the homepage shows mosque photos and highlighted events
Objective	Make sure the carousel and events section loads correctly
Driver/precondition	Admin has uploaded at least one image and one event
Test steps	1. Open the homepage without logging in 2. Look at the top section
Input	None
Expected Results	Image carousel shows photos, upcoming events are listed
Actual Result	Carousel visible, events displayed
Remarks	Pass

8.3. Integration Testing (IT)

We tested how different parts of the system work together in real user flows.

Table 54 Testcase Zakat Request Full Flow

Field	Value
Testcase ID	IT-1
Requirement ID	FR-13, FR-14
Title	Zakat Request End-to-End
Description	User submits request, committee approves, user sees status
Objective	Check the whole flow from submission to final decision

Driver/precondition	User and committee member both have accounts
Test steps	1. User submits request 2. Committee member gets email 3. Committee logs in and approves 4. User checks "My Requests" page
Input	Request details, approval comment
Expected Results	User sees status as "Approved" after committee action
Actual Result	Status updated correctly, visible to user
Remarks	Pass

Table 55 Testcase Multi-Mosque Module Restriction

Field	**Value**
Testcase ID	IT-2
Requirement ID	FR-12, FR-5
Title	Module Restriction for a Mosque
Description	Check that a mosque without the events module does not show event features
Objective	Make sure the module toggle works end to end
Driver/precondition	Mosque Manager has disabled events for a mosque
Test steps	1. Admin of that mosque logs in 2. Checks the sidebar and pages
Input	None
Expected Results	No event management option visible
Actual Result	Events section hidden
Remarks	Pass

8.4. Performance Testing (PT)

We checked the speed and load handling of the most important pages.

Table 56 Testcase Prayer Times Page Load

Field	**Value**
Testcase ID	PT-1

Requirement ID	NFR Performance
Title	Prayer Times Page Load Time
Description	Measure how fast the prayer times appear on the homepage
Objective	Load time should be under 3 seconds
Driver/precondition	System running, internet normal speed
Test steps	1. Open browser DevTools 2. Go to the homepage 3. Record the time for prayer times to appear
Input	None
Expected Results	Under 3 seconds
Actual Result	1.2 seconds
Remarks	Pass

Table 57 Testcase Donation Report Generation

Field	**Value**
Testcase ID	PT-2
Requirement ID	NFR Performance
Title	Donation Report Load Time
Description	Measure time to load the transparency page with 100 records
Objective	Should load in under 5 seconds
Driver/precondition	100 donation records in database
Test steps	1. Log in as admin 2. Go to Donation Transparency page 3. Record time for full list and totals to show
Input	None
Expected Results	Under 5 seconds
Actual Result	2.8 seconds
Remarks	Pass

8.5. Summary

In this chapter, we presented the testing done on the E-Masjid System. We wrote units tests for the main backend functions, functional tests for all user-facing features , integration test and performance test for the most used pages. All test cases are passed and this proves that our system is reliable and fast.

Chapter 7

System Conversion

7. Introduction

System conversion means moving from the old way of doing things to the new system. For our project, the old way is the paper registers and manual methods that most mosques use. The new way is our E-Masjid web application. This chapter explains how we would convert a mosque to our system, how we deployed the application, and what training would be needed.

8.1. Conversion Method

We chose the Pilot Conversion method. This means we would first install the E-Masjid System in only one mosque and run it alongside the paper register for a short time. Both the digital system and the paper register would be used together for about two weeks. This lets the imam and the committee compare the records and check if the system is working correctly. If everything goes well in the pilot mosque, then the system can be used fully, and the paper register can be stopped. Pilot conversion is safe because if any problem comes, the paper register is still there as a backup. It is also cheap because we only need to test in one mosque first. For the multi-mosque feature, the Mosque Manager can add more mosques one by one. Each new mosque would also follow the same pilot method, using both paper and digital together for a short time before fully switching.

8.2. Deployment

We deployed the E-Masjid System on the cloud so that any mosque with internet can access it. We did not use a physical server in a mosque. Below are the steps we followed to deploy the system.

Deployment Steps:

1. **Prepare the code:** We pushed the final code to our GitHub repository, making sure the main branch had the latest working version.
2. **Deploy the backend:** We connected the GitHub repository to Render. Render automatically builds the Node.js backend and starts it. We set the environment variables like database URL, JWT secret, Stripe secret key, and email service credentials in the Render dashboard.
3. **Deploy the frontend:** We connected the React frontend to Vercel. Vercel builds the React app and gives a public URL. We added the backend URL as an environment variable in Vercel so the frontend knows where to send API requests.
4. **Set up the database:** We used MongoDB Atlas, a cloud database. We created a new cluster

and added the connection string to the backend environment. We also enabled automatic daily backups in Atlas.

5. **Check the deployment:** After both frontend and backend were deployed, we opened the Vercel URL in a browser and tested the main features: prayer times loading, user registration, and login. Everything worked.

7.2.1 Data Conversion

When a mosque starts using the E-Masjid System, there is very little data to convert because the old system is paper based. The mosque admin would need to enter some initial data manually. We suggest these steps:

1. **Extract key data:** The admin takes the current paper register and writes down the important things that should be in the system: recent donations, current expenses, upcoming events, and the usual prayer times.
2. **Clean the data:** The admin checks for any missing amounts, unclear names, or wrong dates in the paper records. They fix these before entering.
3. **Enter data into the system:** The admin logs in and manually adds:
   The prayer times for the current week.
   Any cash donations from the last few days.
   The current expenses.
   Any upcoming events and announcements.
4. **Verify:** After entering, the admin checks the financial reports page to see if the balance matches what the paper register shows.

7.2.2 Training

We prepared a simple user manual for two main use cases so that mosque staff can learn the system quickly.

Training for: Admin Recording a Cash Donation

1. Go to the admin login page: your-url/admin/login.
2. Enter your email and password, click Login.
3. On the dashboard, click the "Record Donation" card.
4. A form opens. Enter the donor name, select the donation type (Zakat, Sadaqah, or Mosque

Fund), enter the amount, and pick the date.

5. If the donor wants to be anonymous, tick the "Anonymous" checkbox.
6. Click "Save Donation".
7. A green message appears: "Donation recorded successfully."
8. To check, go to the Transparency page and see the donation in the list.

Training for: Community Member Submitting a Zakat Request

1. Open the website and log in with your email and password.
2. On your dashboard, click "Request Zakat Help".
3. Fill in your name, phone number, reason for needing help, and the amount you need.
4. Click "Submit Request".
5. A green message appears: "Request submitted. The committee will review it."
6. To check the status later, go to "My Requests" from the menu. You will see your request and its status.

8.3. Post Deployment Testing

After deploying the system, we ran a set of tests again to make sure everything works correctly in the live environment. We did the following:

1. Opened the homepage and checked that prayer times load quickly.
2. Registered a new user and logged in.
3. Made an online donation using Stripe test mode and checked that it appeared in the records.
4. Submitted a Zakat request and checked that the committee member received the email.
5. Logged in as a committee member and approved a request. Checked that the requester could see the updated status.
6. Logged in as the Mosque Manager and created a new mosque with limited modules. Checked that the admin of that mosque could only see the assigned features.

All tests passed. We also used the browser developer tools to check for any errors in the console.

8.4. Challenges

We faced a few challenges while deploying and converting the system. Here are the main ones and how we solved them.

1. **Environment variables not loading:** When we first deployed the backend on Render, it could not connect to MongoDB because the environment variable for the database URL was not set correctly. We fixed this by adding all the required variables in the Render dashboard and redeploying.
2. **Email service not working:** Nodemailer was not sending emails because Gmail blocked the login from an unknown app. We solved this by creating an "App Password" in the Gmail settings and using that instead of the normal password.
3. **Stripe key mismatch:** The frontend was using the test publishable key, but the backend was using the live secret key by mistake. Payments were failing. We fixed this by making sure both frontend and backend use the test keys for development.
4. **Committee emails going to spam:** Some test emails to committee members went to the spam folder. We added a clear subject line and a short message body to reduce the chance of this happening. For a real deployment, we would use a professional email service.
5. **Training hesitation:** When we showed the system to one mosque member, he was worried about making mistakes. We solved this by showing him that all actions can be corrected, and the paper register is still there during the pilot phase.

8.5. Summary

In this chapter, we described the system conversion process for the E-Masjid System. We chose pilot conversion to safely move from paper registers to the digital system. We deployed the backend on Render, the frontend on Vercel, and the database on MongoDB Atlas. Data conversion is done manually by the mosque admin entering existing records. We provided a simple training manual for key tasks like recording donations and submitting Zakat requests. Post-deployment testing confirmed that the system works correctly in the live environment. We listed the challenges we faced during deployment and how we solved them. The system is now ready for a real mosque to use.

Chapter 8

Conclusion

8. Introduction

This chapter wraps up the E-Masjid System project. We look back at the objectives we set in Chapter 1, check if we achieved them and show how each requirement connects to the design, code, and testing. We also discuss what the project means and what could be done in the future.

8.1. Evaluation

Here we list all the objectives we wrote in Chapter 1 and mark whether they are completed.

Table 58 Evaluation

Objectives	Status
To build a web-based mosque management system using MERN stack.	Completed
To manage donations transparently, track expenses, and allow online donations.	Completed
To display daily prayer times and special timings for Jummah and Ramadan.	Completed
To create an event section for Islamic classes, charity, and community programs.	Completed
To provide an online booking system for Nikah registrar services.	Completed
To assign different access levels for admin, religious scholars, and community members.	Completed
To implement a Mosque Manager role that can control multiple mosques and assign modules.	Completed
To build a Zakat and Sadaqah request system with committee review.	Completed
To allow anonymous donations and display a top donors section.	Completed
To design a promotional homepage section with mosque photos and event highlights.	Completed

8.2. Traceability Matrix

This table shows how every requirement is linked to a design part, a code file, and a test case.

Table 59 Traceability Matrix

Requirement ID	Requirement Description	Design Specification	Code	Test ID
FR-1	User registration and login with roles	User Authentication Module	authController.js, User.js	UT-1

FR-2	Record donations with anonymous option	Donation Management Module	donationController.js, Donation.js	UT-2
FR-3	Record mosque expenses	Expense Management Module	expenseController.js, Expense.js	-
FR-4	Show donation and expense reports	Financial Reporting Module	reportController.js	PT-2
FR-5	Event and announcement management, promotional section	Event & Announcement Manager	eventController.js, announcementController.js	FT-5
FR-6	Book Nikah services	Nikah Booking Module	nikahController.js, NikahBooking.js	-
FR-7	Manage prayer times	Prayer Times module	prayerController.js, PrayerTime.js	PT-1
FR-8	Online donation system	Online Payment Module	paymentController.js	FT-1
FR-9	Password reset through email	User Authentication Module	authController.js	-
FR-10	Check Nikah booking status	Nikah Booking Module	nikahController.js	-
FR-11	Create scholar accounts User	Authentication Module	userController.js	-
FR-12	Multi-mosque management by Mosque Manager	Mosque Management Module	mosqueController.js, Mosque.js	FT-4, IT-2
FR-13	Zakat/Sadaqah request submission	Zakat Request Module	zakatController.js, ZakatRequest.js	UT-3, FT-2, IT-1
FR-14	Committee review of Zakat requests	Committee Review Module	zakatController.js	FT-3, IT-1

8.3. Conclusion

The E-Masjid System was built to solve real problems that mosques in Pakistan face every day. We started with a simple idea to bring transparency to donation records and make mosque management easier. Over two semesters, we added more features based on feedback from our supervisor and our own thinking. Now the system is a complete platform. The mosque admin can manage prayer times, donations, expenses, events, announcements, and Nikah bookings. Community members can stay connected, donate online, book Nikah services, and even request Zakat help. The Mosque Manager can oversee multiple mosques and control which features each one gets. The Zakat request system brings

fairness and record-keeping to a process that was previously informal. Small but important features like anonymous donations and top donors add privacy and motivation. We built the system using the MERN stack, kept the interface simple so anyone can use it, and tested it properly. The project taught us a lot about full-stack development, but more importantly, it taught us how to listen to real users and build something useful. We hope that one day a mosque actually uses this system and benefits from it.

8.4. Future Work

There are still things that can be added or improved in the E-Masjid System in the future.

1. **SMS notifications:** Right now the system only sends emails for Zakat requests and password resets. In the future, SMS alerts can be added so that people without email can also get notified.
2. **Mobile app:** The website works on mobile phones, but a dedicated Android or iOS app would make it even easier for some users.
3. **Urdu language support:** The interface is in English and simple Roman Urdu. A full Urdu translation would help elderly users who are not comfortable with English.
4. **Donation receipt printing:** The system could generate a PDF receipt that users can download or print after donating.
5. **More payment options:** Along with Stripe, local Pakistani payment gateways like JazzCash or EasyPaisa could be integrated so people can donate directly from their mobile wallets.

References

1. World Wide Web

[1] Meta Platforms Inc., "React Documentation,". Available: https://react.dev/.

[2] System Design Specification (SDS) "YouTube lecture used to understand Software / System Design Specification". Available: https://www.youtube.com/watch?v=hOH7eD9NJgY .

[3] Class Diagram (UML), "YouTube tutorial used to learn UML Class Diagrams". Available: https://www.youtube.com/watch?v=6XrL5jXmTwM .

[4] Component Diagram, "YouTube tutorial used to understand Component Diagrams". Available: https://youtu.be/CW9Ts2qLfEI?si=QXdUxthCEo-fcigr .

[5] Sequence Diagram, "YouTube tutorial used to learn Sequence Diagrams using draw.io". Available: https://www.youtube.com/watch?v=rsTWufuP328 .

[6] OpenJS Foundation, "Node.js Documentation,". Available: https://nodejs.org/en/docs/.

[7] MongoDB Inc., "MongoDB Manual,". Available: https://www.mongodb.com/docs/manual/.

[8] The guide for using Mongoose with a database., "Mongoose Documentation," Available: https://mongoosejs.com/docs/.

[9] Stripe Inc., "Stripe API Reference,". Available: https://stripe.com/docs/api.

[10] Auth0, "Introduction to JSON Web Tokens,". Available: https://jwt.io/introduction.

[11] Draw.io User Manual for UML Diagrams. Available: https://www.drawio.com/.